

REDES NEURAIS E DEEP LEARNING

Aula 03 - Perceptron Multicamadas (*Multilayer Perceptron* - MLP)

Prof. Rafael G. **Mantovani**

Roteiro



- 1** Introdução
- 2** Multilayer Perceptron
- 3** Formalização
- 4** Treinamento
- 5** Backpropagation
- 6** Referências

Roteiro

- 1** Introdução
- 2** Multilayer Perceptron
- 3** Formalização
- 4** Treinamento
- 5** Backpropagation
- 6** Referências

Introdução



Introdução

Perceptron

1958

Introdução



MLP
(Backpropagation)

A horizontal timeline with a black arrow pointing to the right. A light blue circle is positioned on the left, and a dark blue circle is positioned further to the right. The text 'MLP (Backpropagation)' is centered above the dark blue circle, and the year '1986' is centered below it.

1986

Introdução

- **MultiLayer Perceptron** (Rosenblatt)
 - Algoritmo de Retropropagação (**backpropagation**) [Rumerhalt & et al]
 - Supera as limitações práticas do Perceptron (simples)

MLP
(Backpropagation)

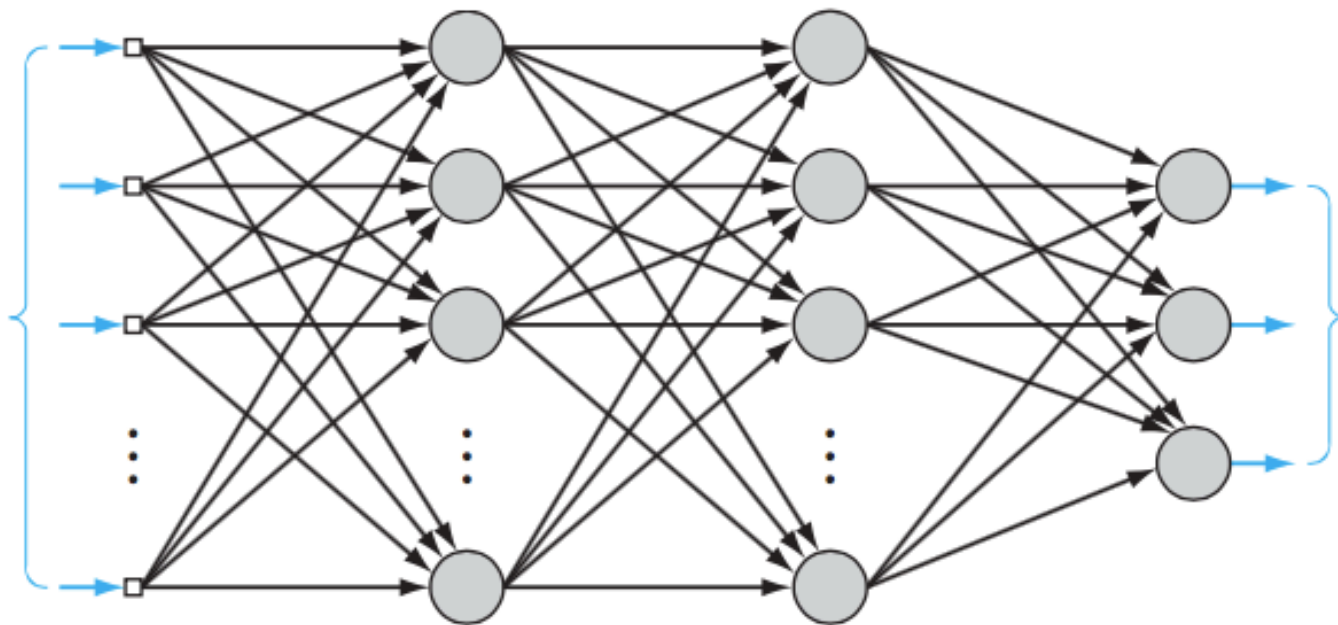
1986

Introdução

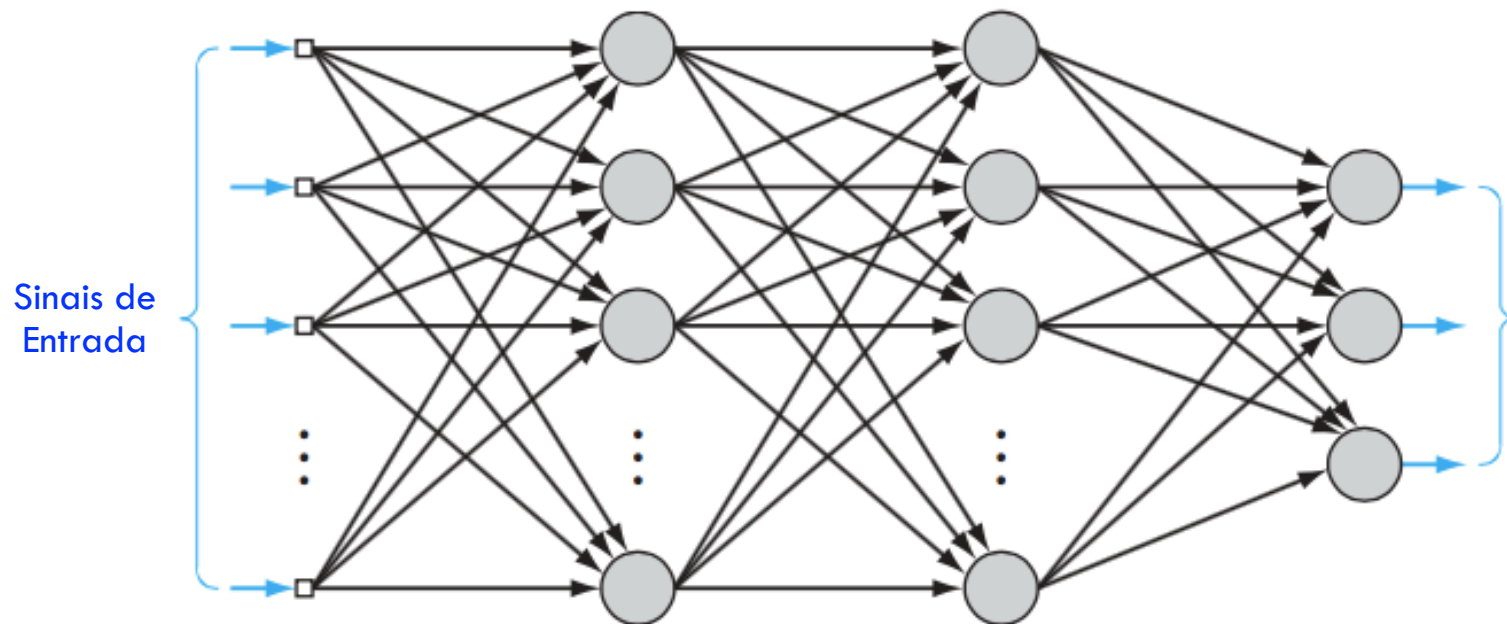
- **Multilayer Perceptron:**

- neurônios possuem uma função de ativação **não-linear e diferenciável**
- contém uma ou mais camadas escondidas
- a rede possui alto grau de conectividade

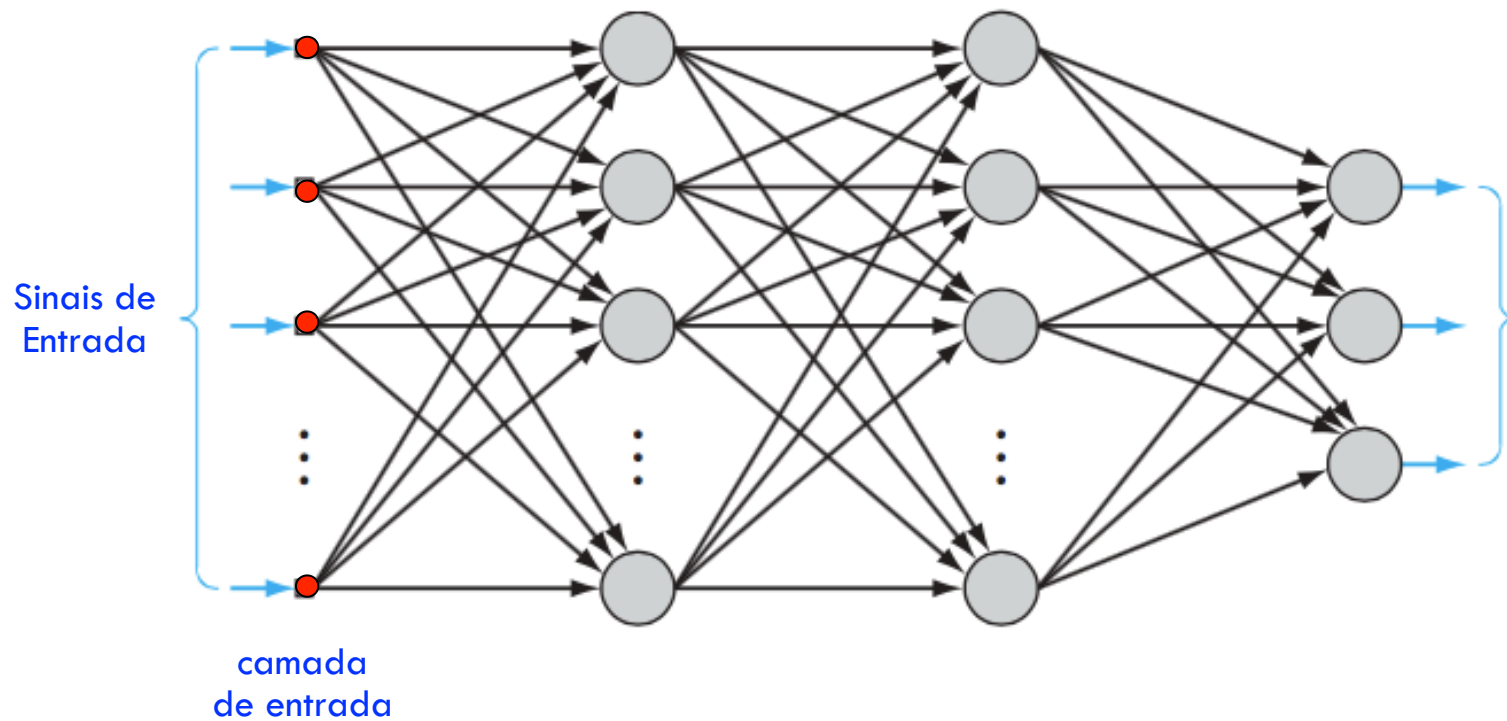
Introdução



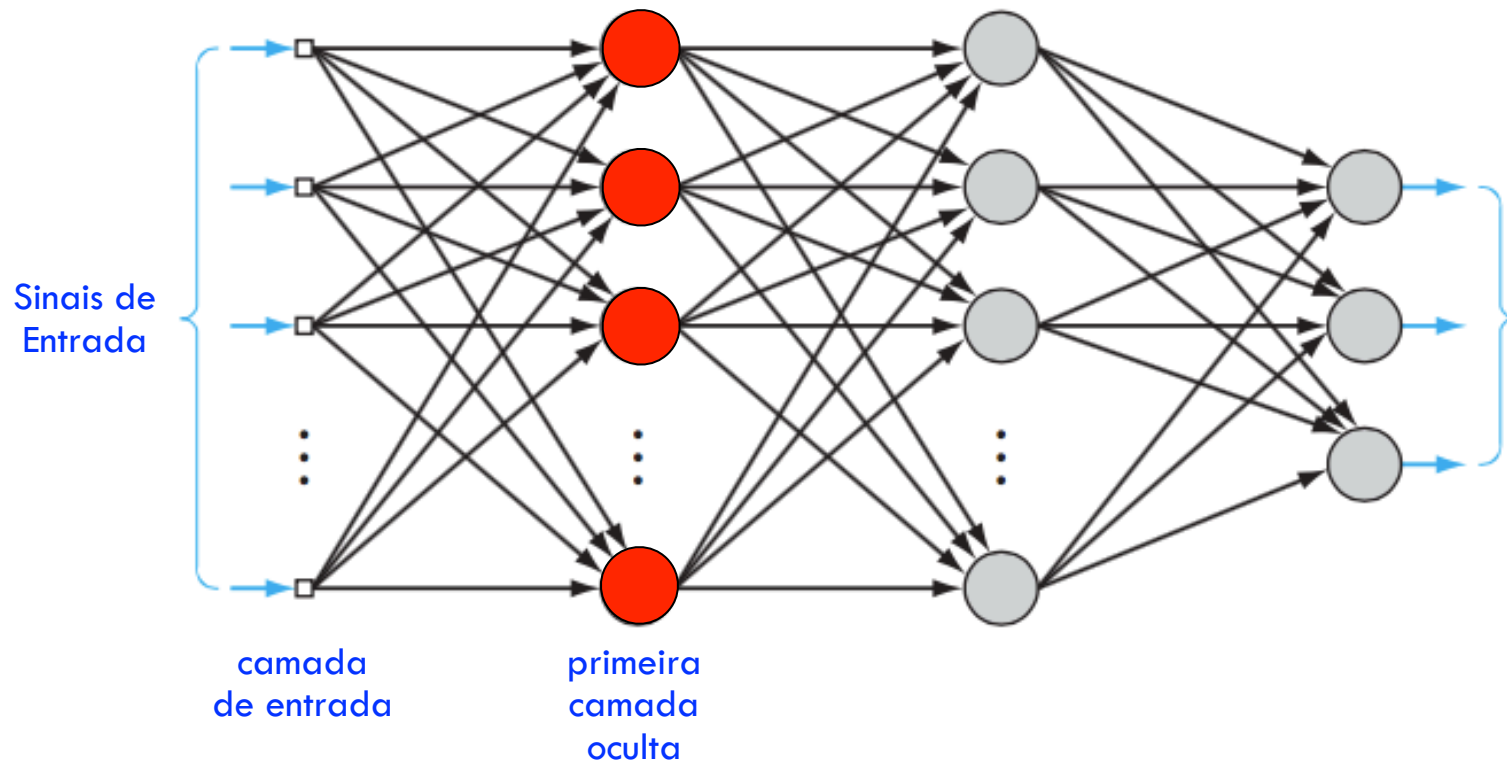
Introdução



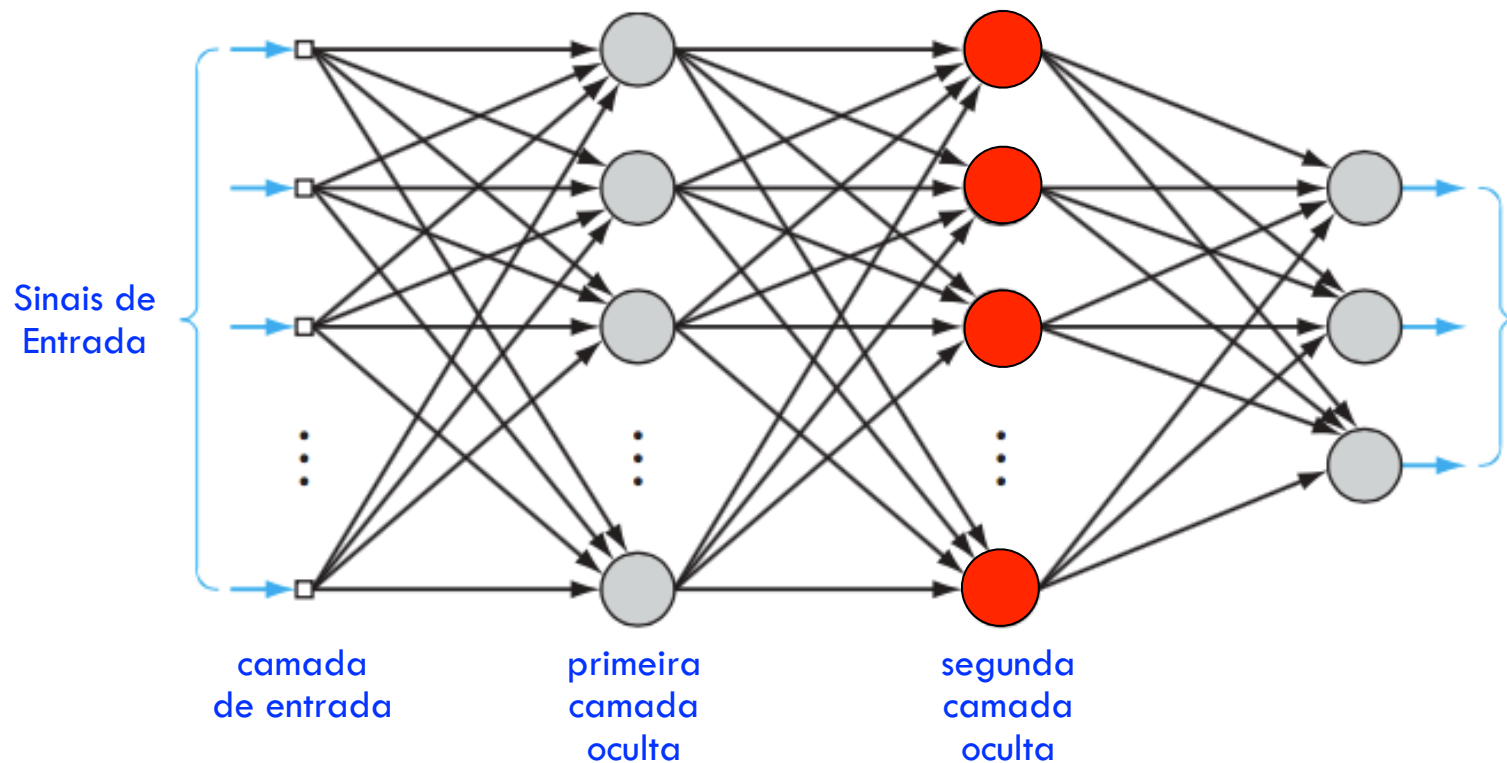
Introdução



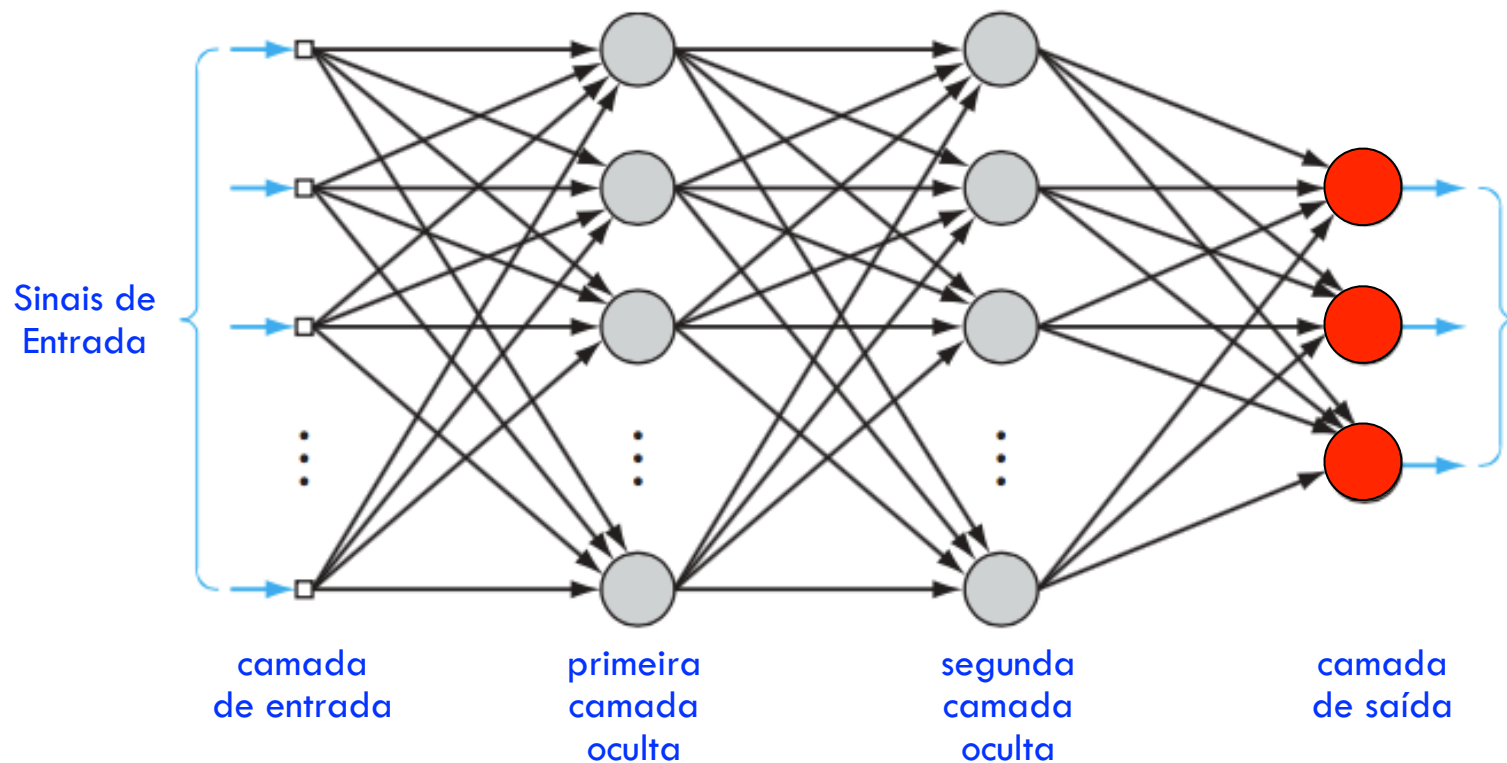
Introdução



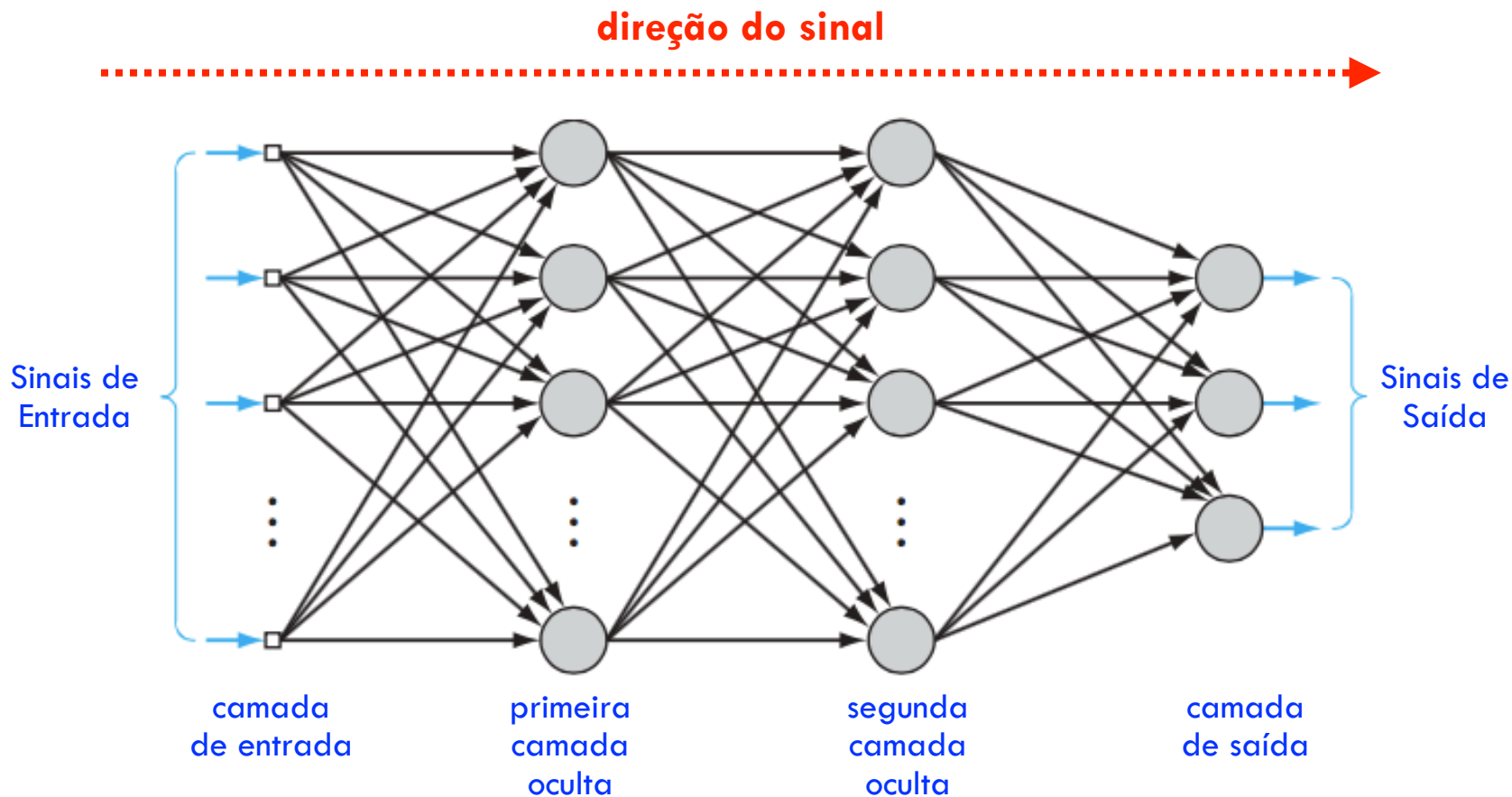
Introdução



Introdução



Introdução



Introdução



- **Deficiências:**

Introdução

□ Deficiências:

1

A **análise teórica** é **difícil**. Há muitas conexões e funções não-lineares

Introdução

□ Deficiências:

1

A **análise teórica** é **difícil**. Há muitas conexões e funções não-lineares

2

Muitos neurônios: **difícil** de **visualizar** o processo de aprendizado

Introdução

□ Deficiências:

- 1 A **análise teórica** é **difícil**. Há muitas conexões e funções não-lineares
- 2 Muitos neurônios: **difícil** de **visualizar** o processo de aprendizado
- 3 **Aprendizado** é **difícil**. Há um espaço muito maior de funções, e mais representações dos padrões de entrada

Roteiro

- 1 Introdução
- 2 Multilayer Perceptron
- 3 Formalização
- 4 Treinamento
- 5 Backpropagation
- 6 Referências

Multilayer Perceptron



Como **aprender?**

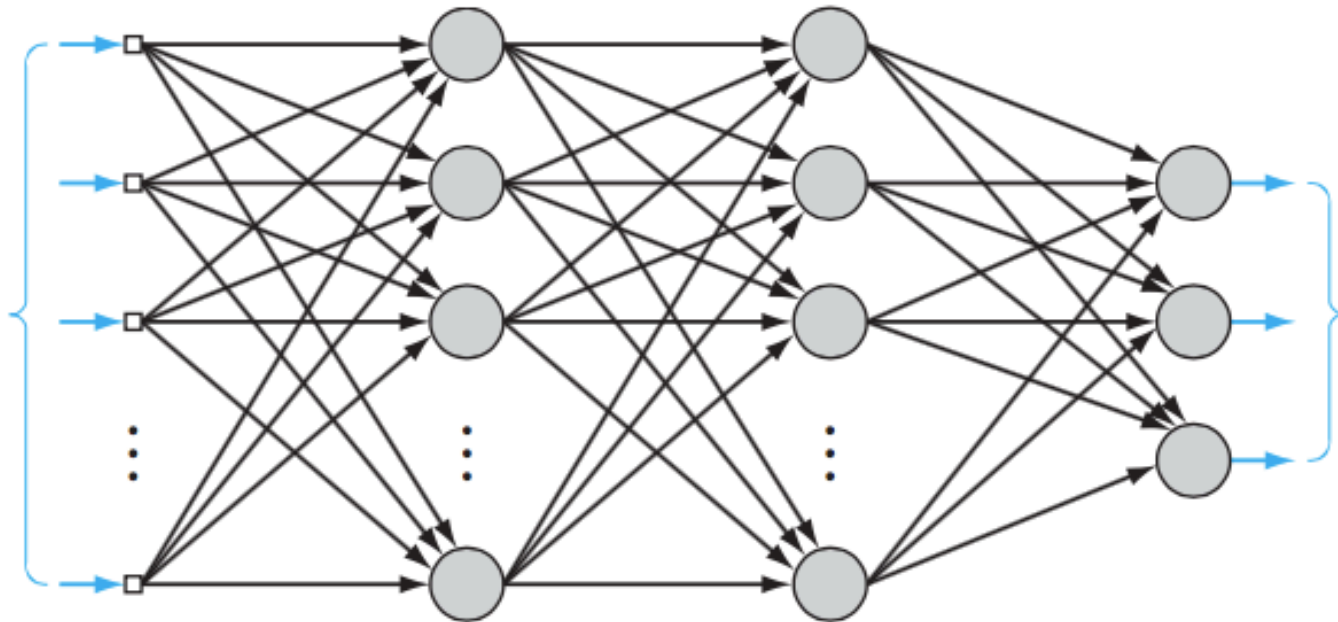
Multilayer Perceptron



- ***Backpropagation***

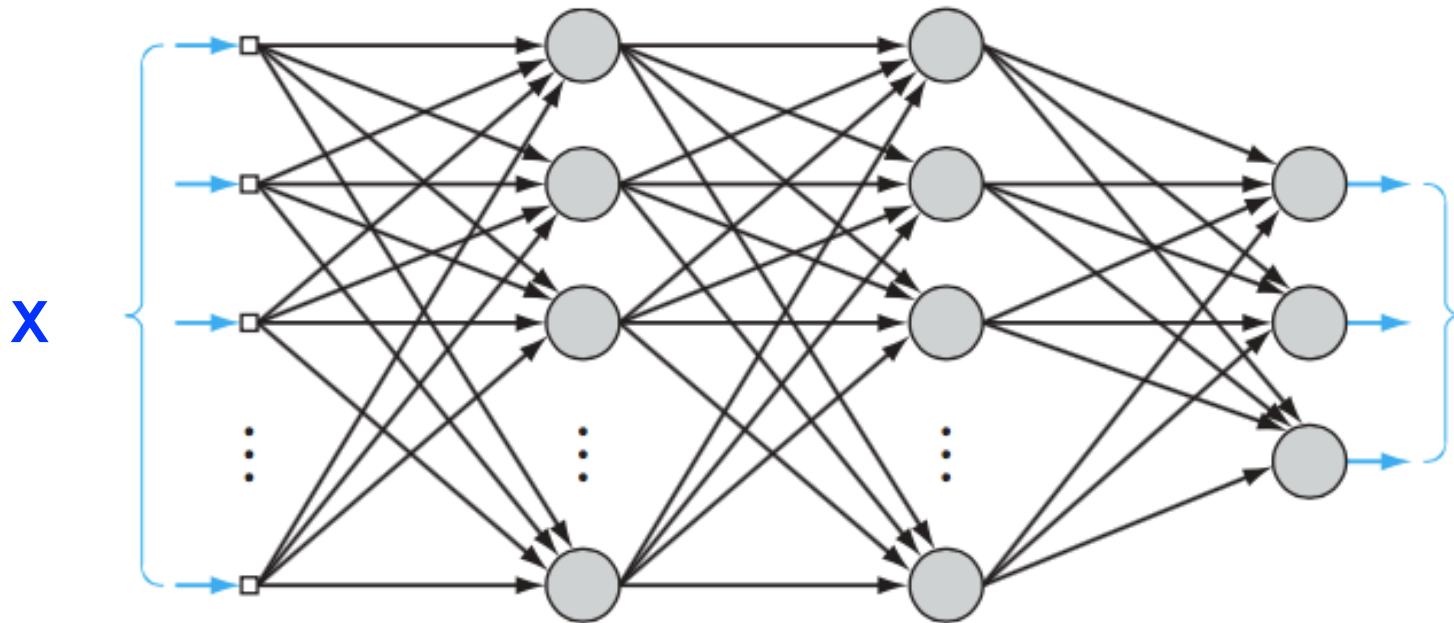
Multilayer Perceptron

- **Backpropagation**



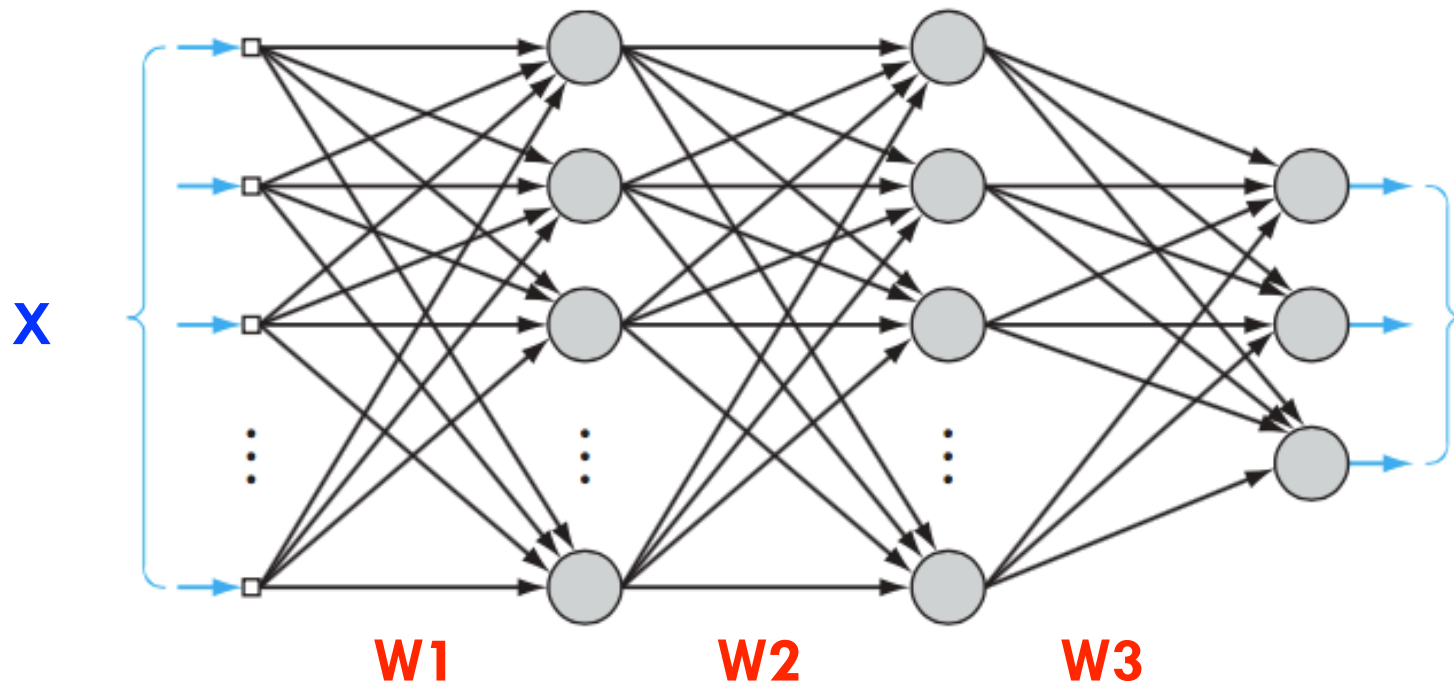
Multilayer Perceptron

- **Backpropagation**



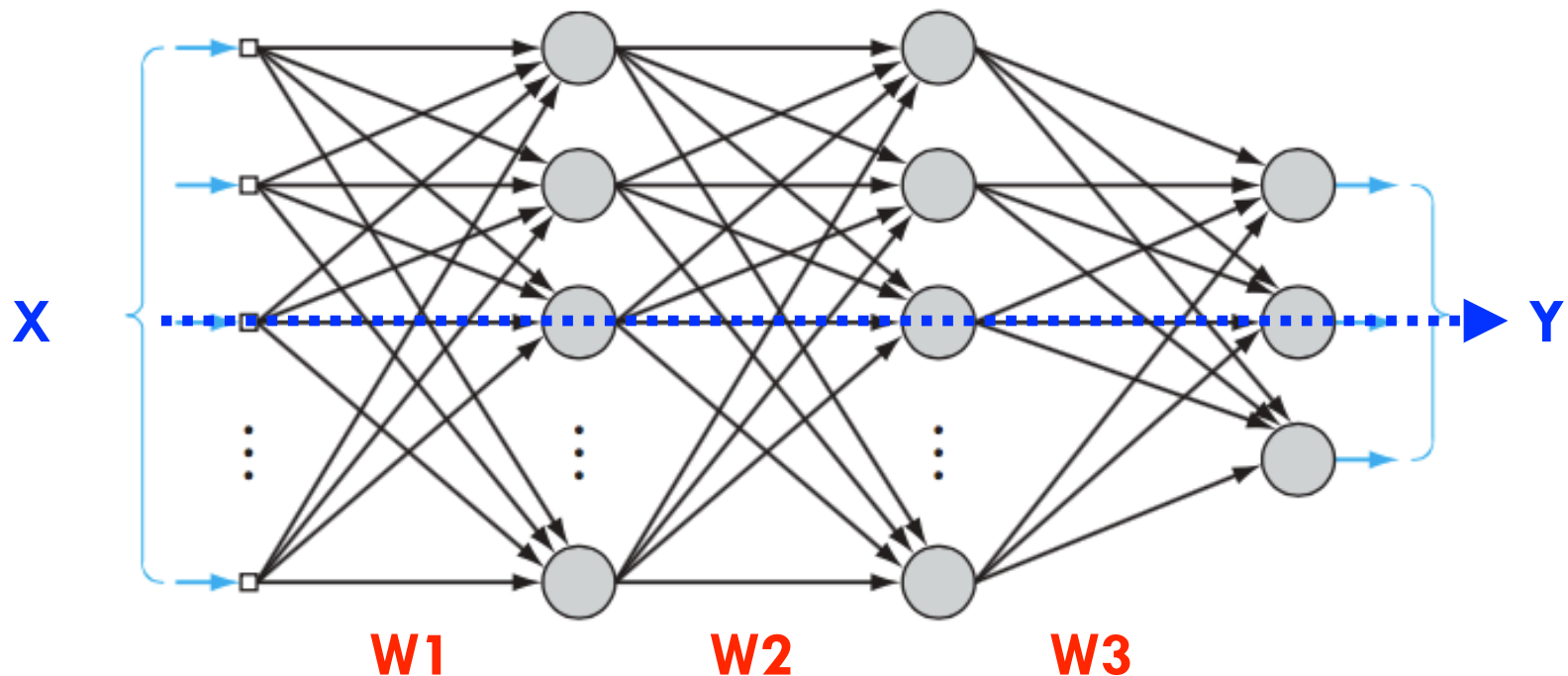
Multilayer Perceptron

- **Backpropagation**



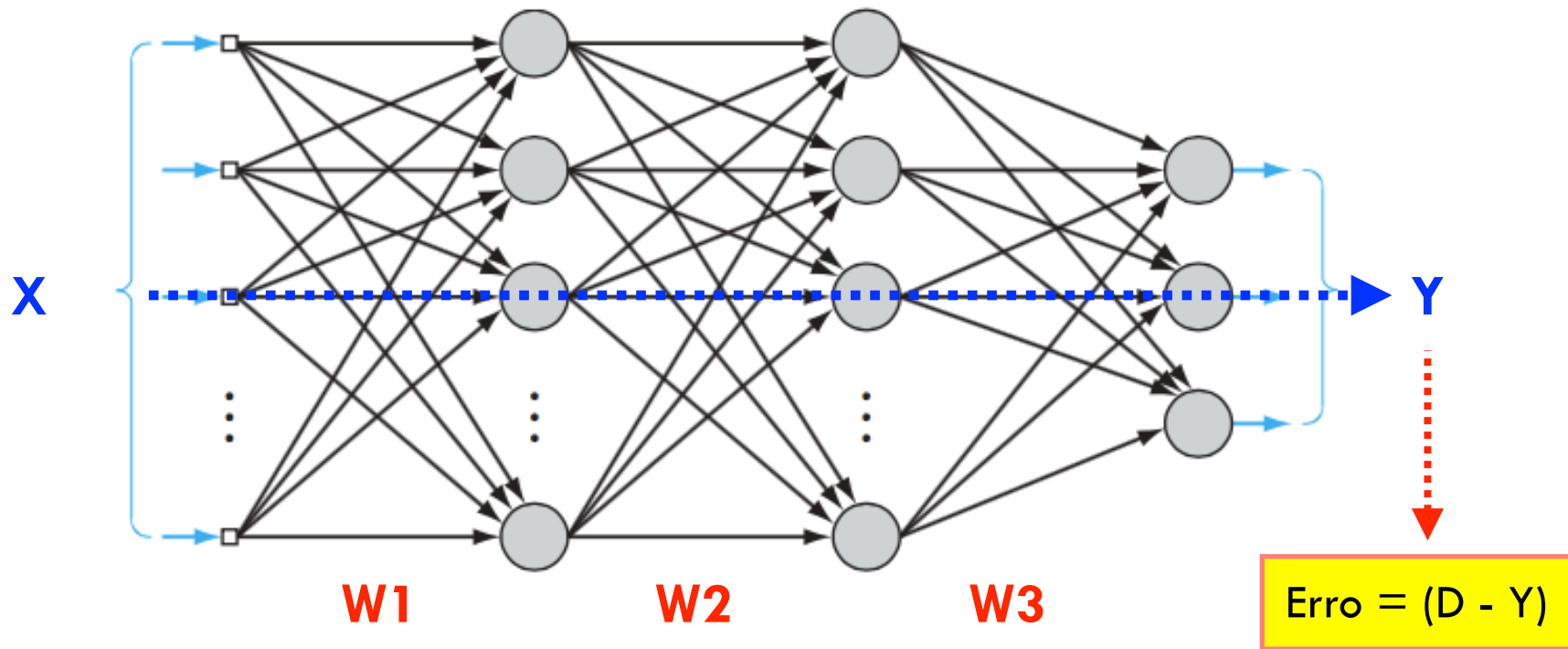
Multilayer Perceptron

- **Backpropagation**



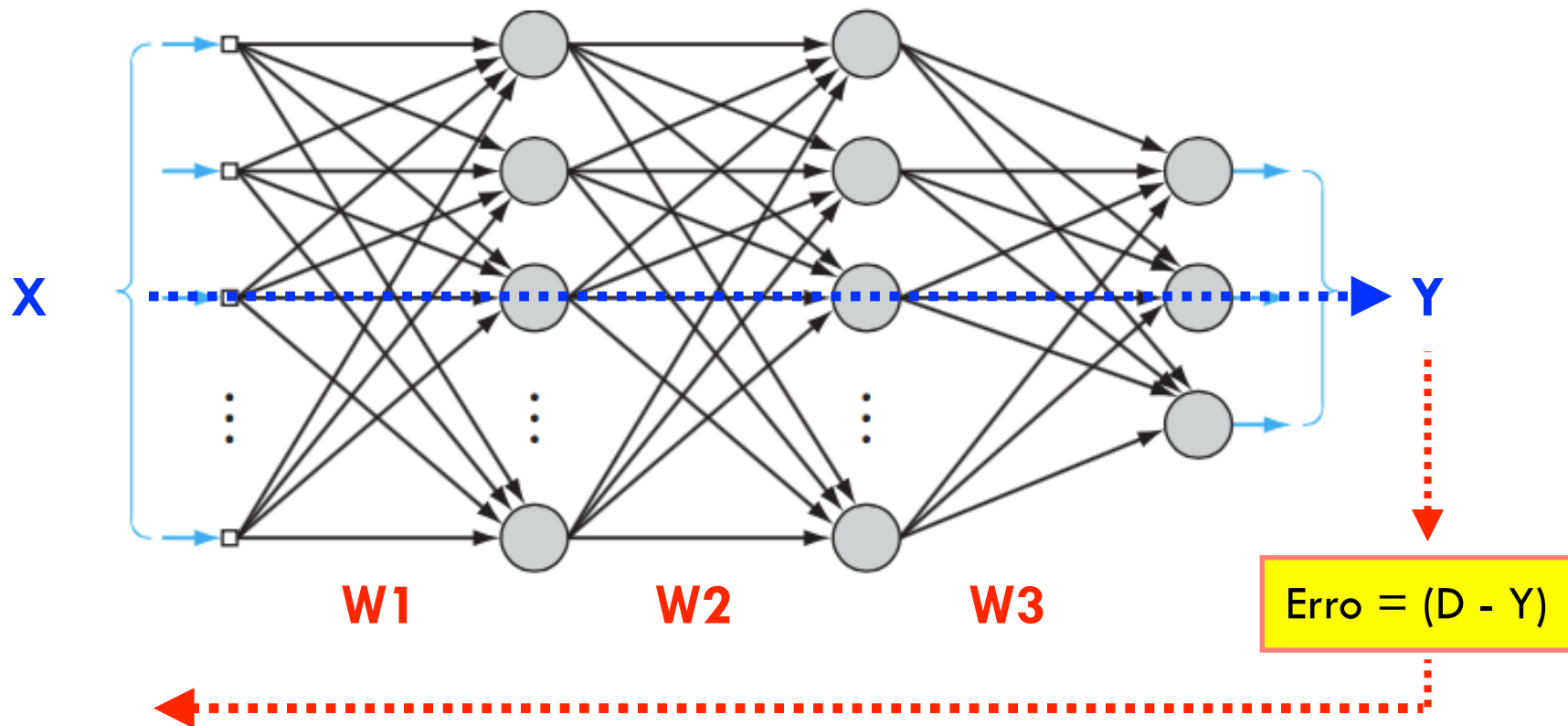
Multilayer Perceptron

- **Backpropagation**



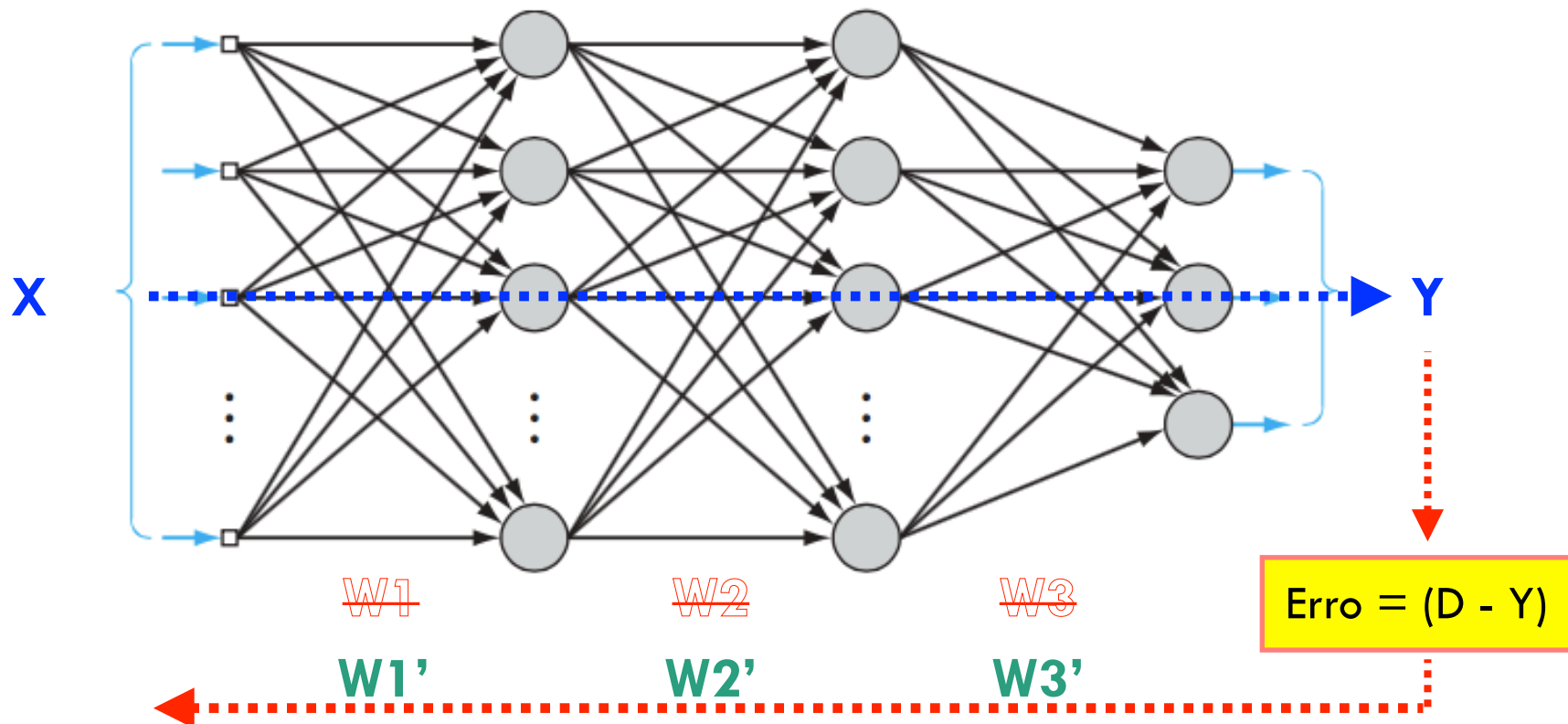
Multilayer Perceptron

- Backpropagation**



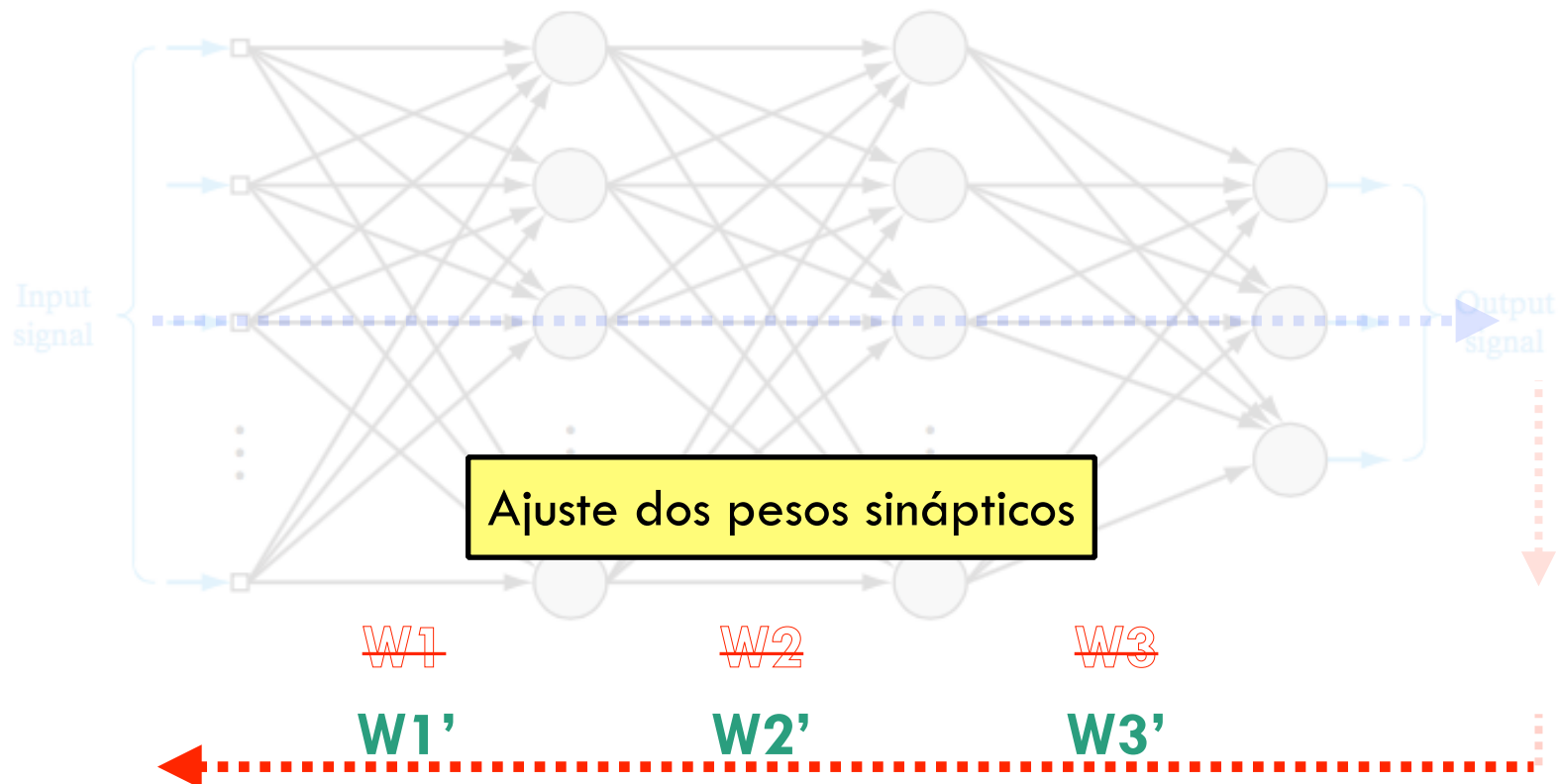
Multilayer Perceptron

- Backpropagation**



Multilayer Perceptron

- **Backpropagation**



Multilayer Perceptron



- **2 Fases:**

Multilayer Perceptron

- **2 Fases:**

- **Forward:** propagação do sinal

Entrada → Camadas Ocultas → Saída
Estimativa do erro

Multilayer Perceptron

□ 2 Fases:

- **Forward:** propagação do sinal

Entrada → Camadas Ocultas → Saída
Estimativa do erro

- **Backward:** sinal de erro é retropropagado

Entrada ← Camadas Ocultas ← Saída
Ajustes sinápticos

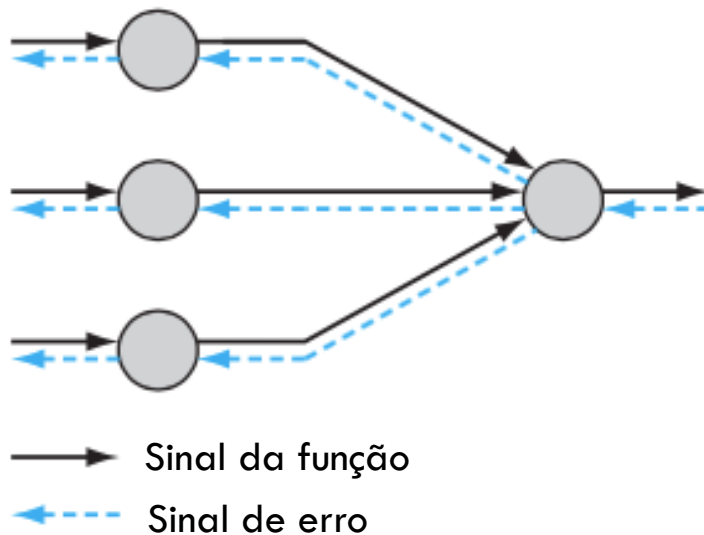
Multilayer Perceptron



- Neurônios da camada oculta, executam dois cálculos:

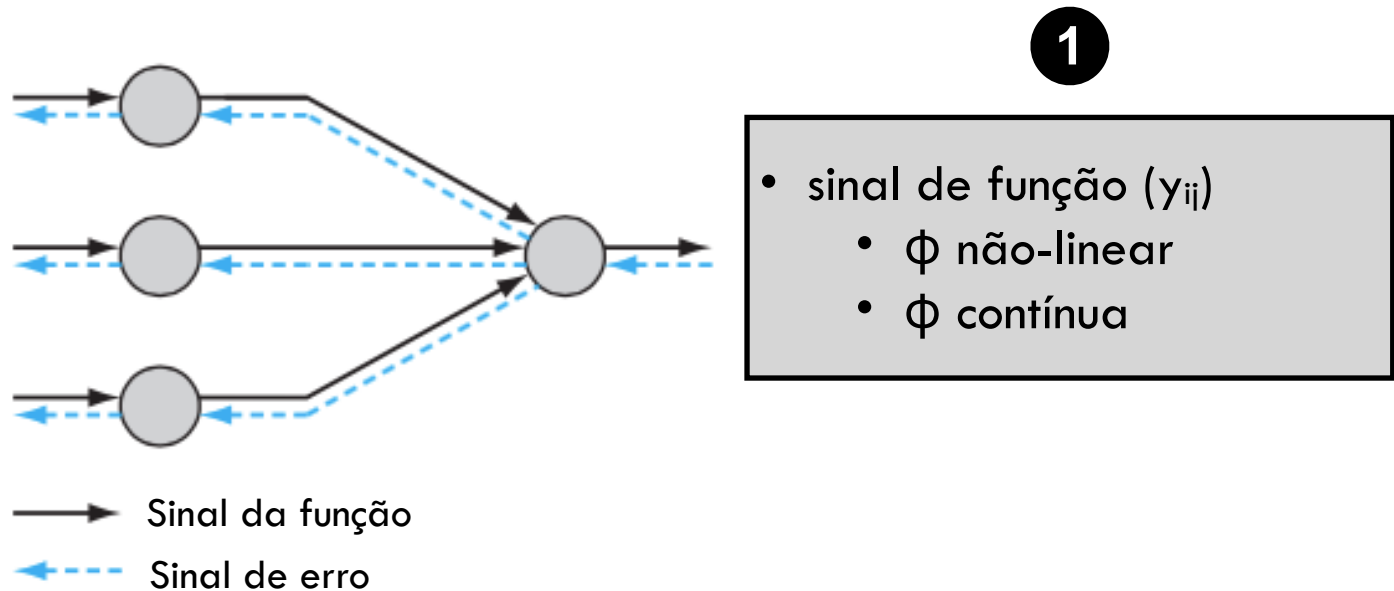
Multilayer Perceptron

- Neurônios da camada oculta, executam dois cálculos:



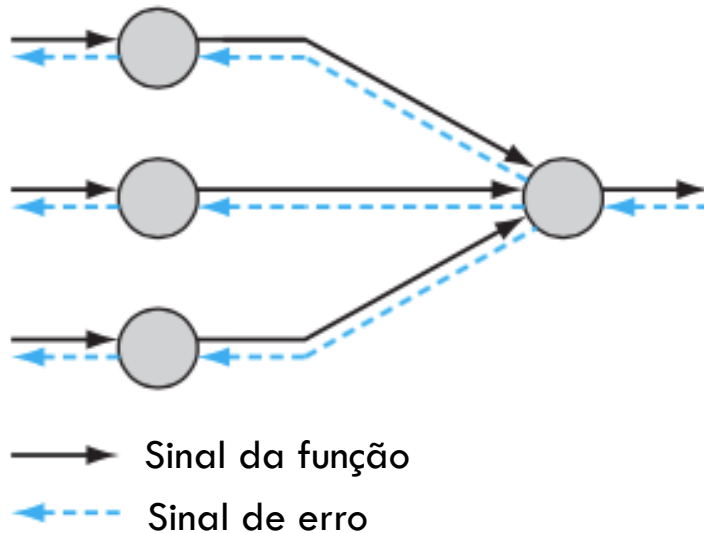
Multilayer Perceptron

- Neurônios da camada oculta, executam dois cálculos:



Multilayer Perceptron

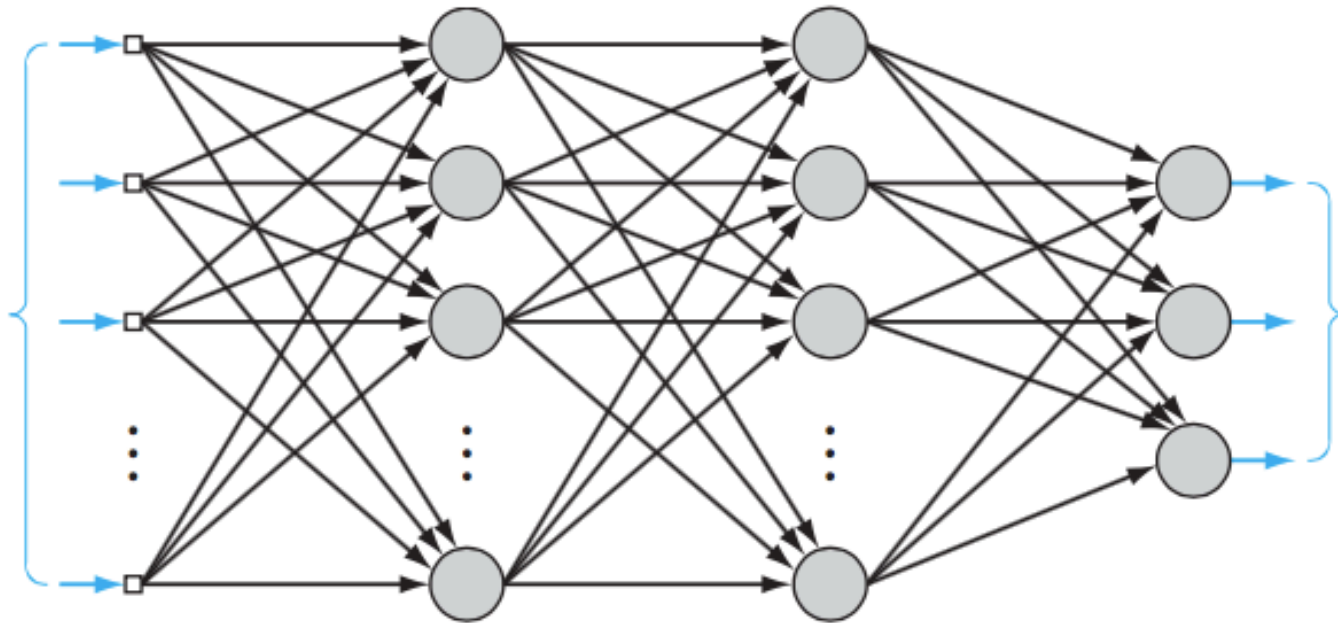
- Neurônios da camada oculta, executam dois cálculos:



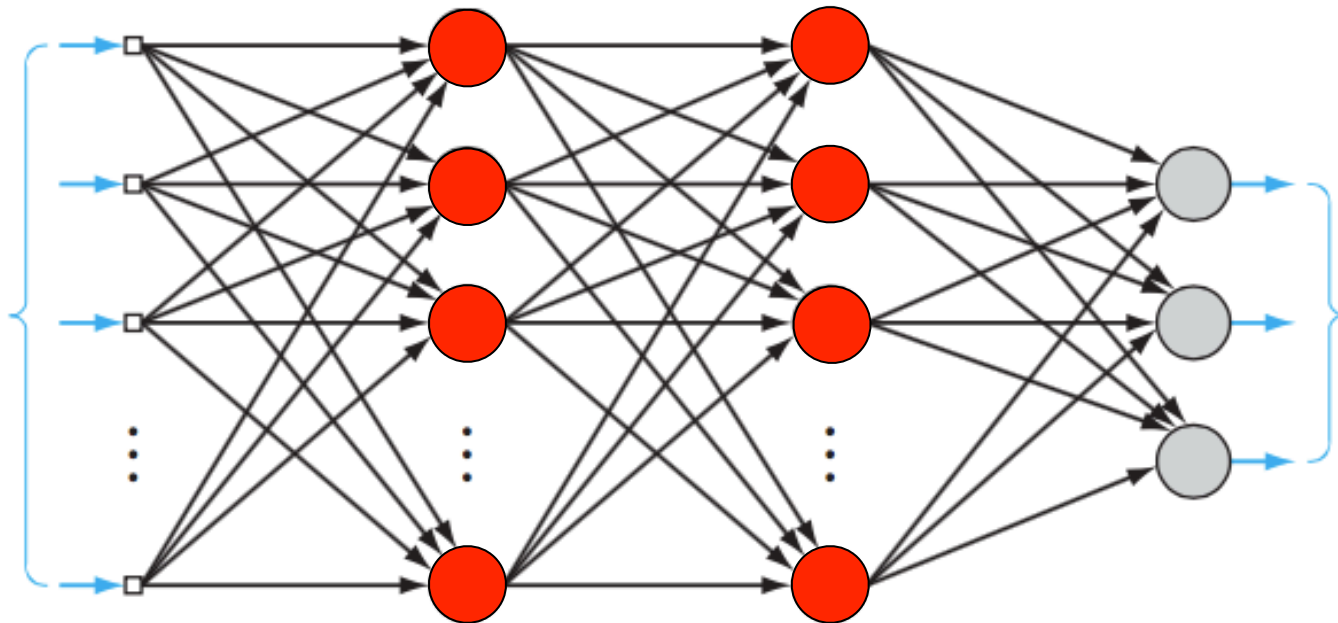
2

- estimativa do vetor gradiente (δ_{ij})
 - gradiente da superfície de erro
 - **retropropagação**

Multilayer Perceptron

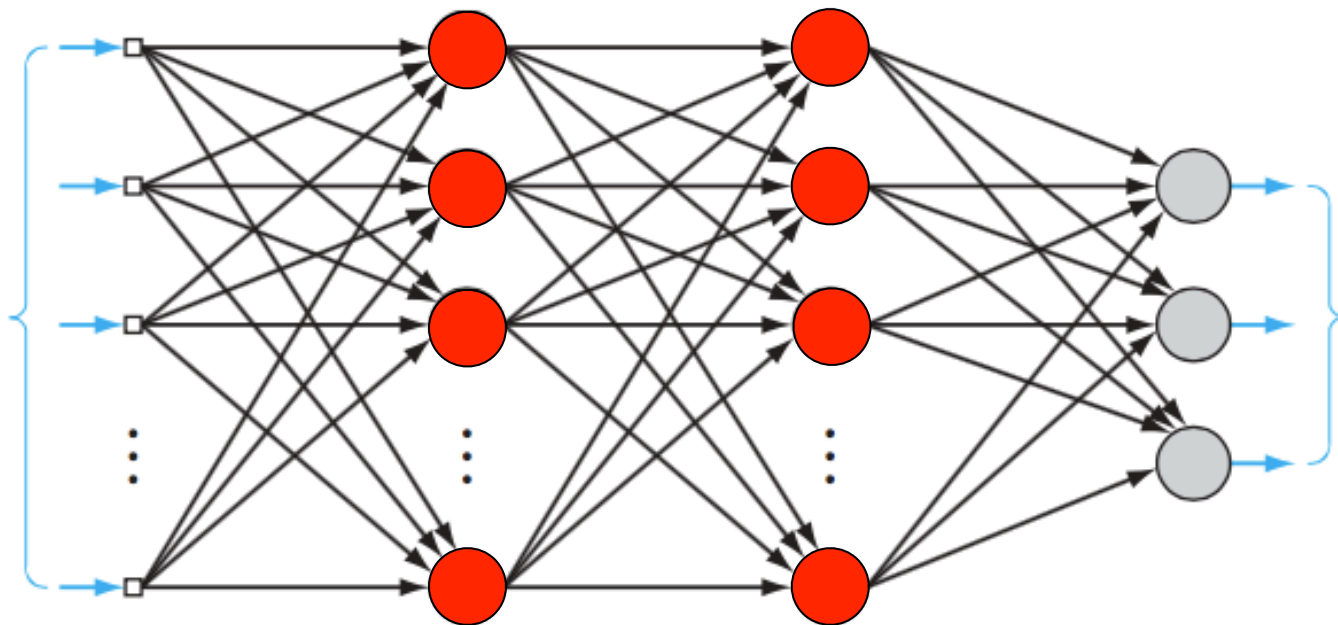


Multilayer Perceptron



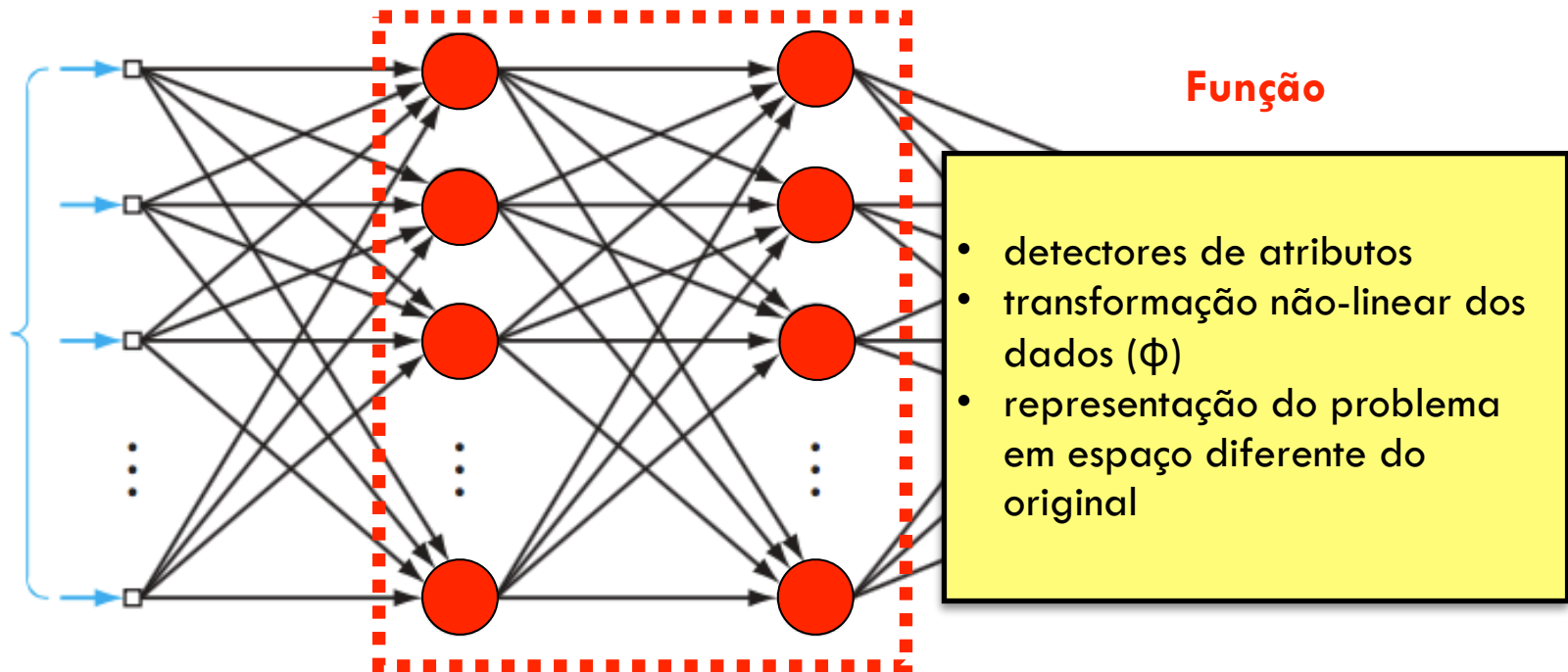
Multilayer Perceptron

- Neurônios das camadas ocultas



Multilayer Perceptron

- Neurônios das camadas ocultas



Multilayer Perceptron



- **O que as camadas intermediárias formam?**

Multilayer Perceptron

- **O que as camadas intermediárias formam?**

1 camada → **linhas** retas no espaço de decisão

Multilayer Perceptron

□ O que as camadas intermediárias formam?

1 camada → **linhas** retas no espaço de decisão

2 camada → combina as linhas da camada anterior em **regiões convexas**

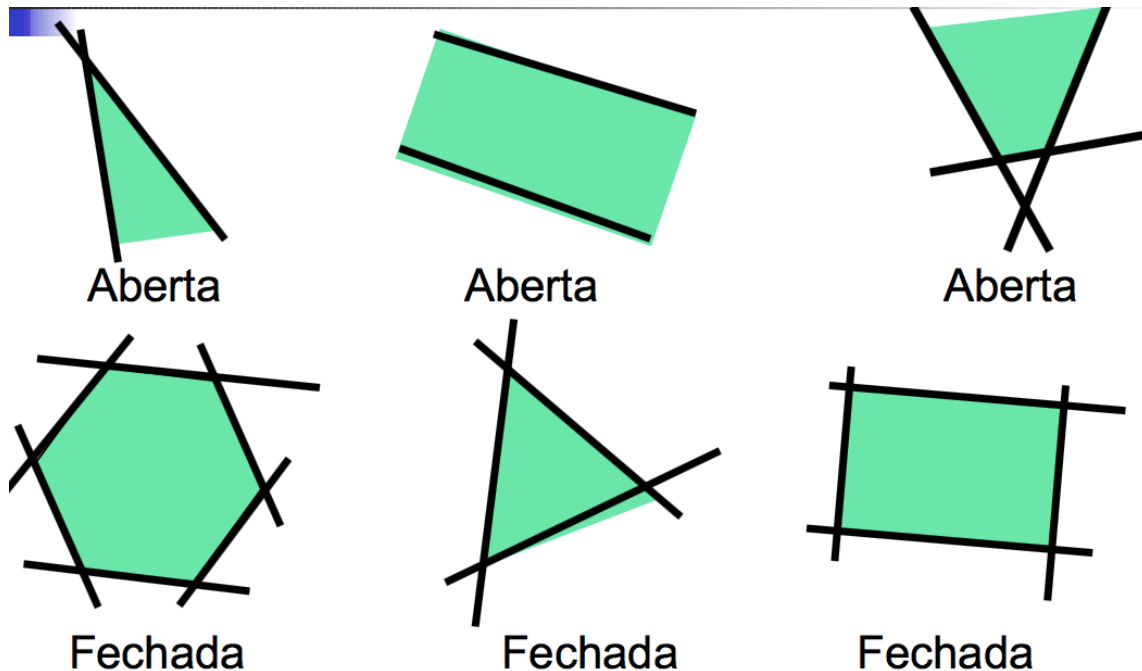
Multilayer Perceptron

□ O que as camadas intermediárias formam?

1 camada → **linhas** retas no espaço de decisão
2 camada → combina as linhas da camada anterior em **regiões convexas**
3 camada → combina as regiões convexas produzindo **formatos abstratos**
...

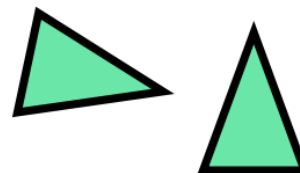
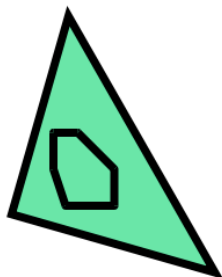
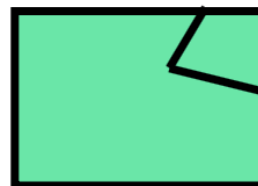
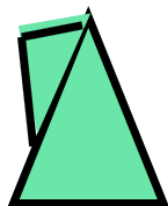
Regiões convexas

□ Combinações de hiperplanos

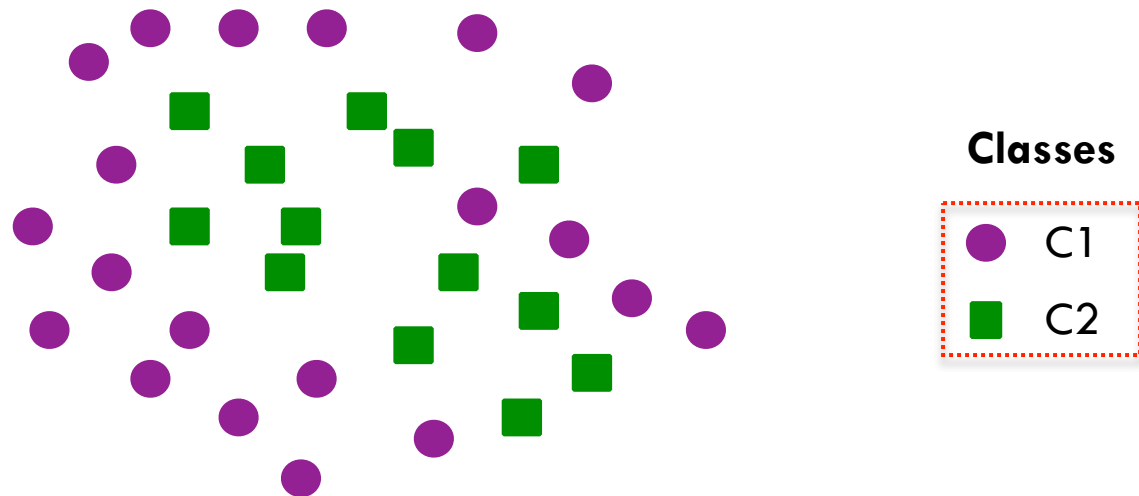


Figuras convexas

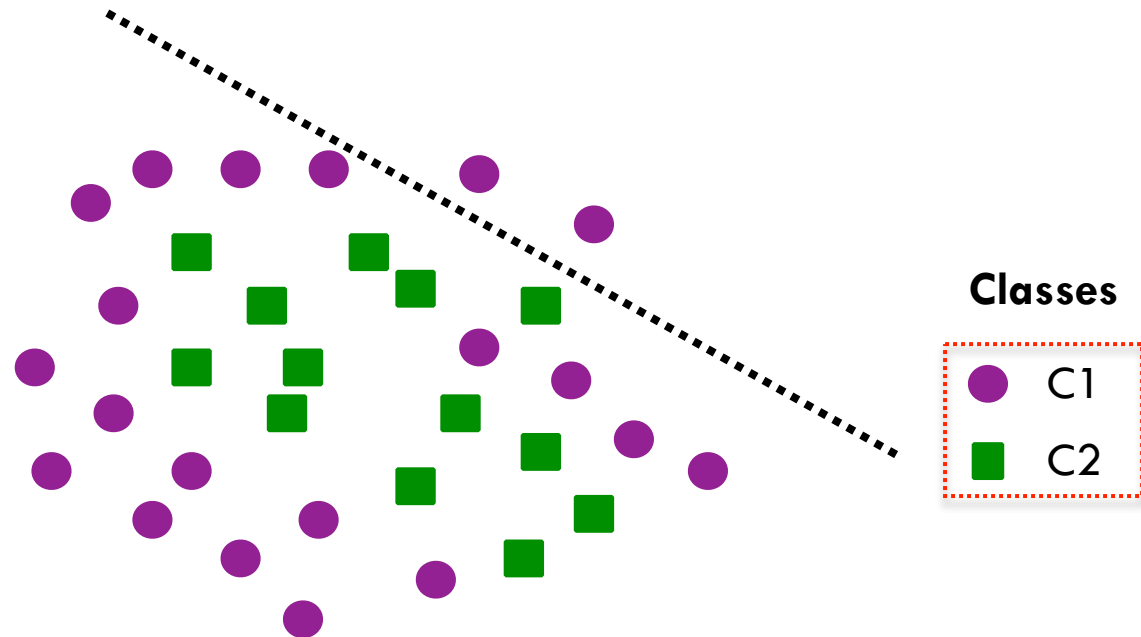
□ Combinações de regiões convexas



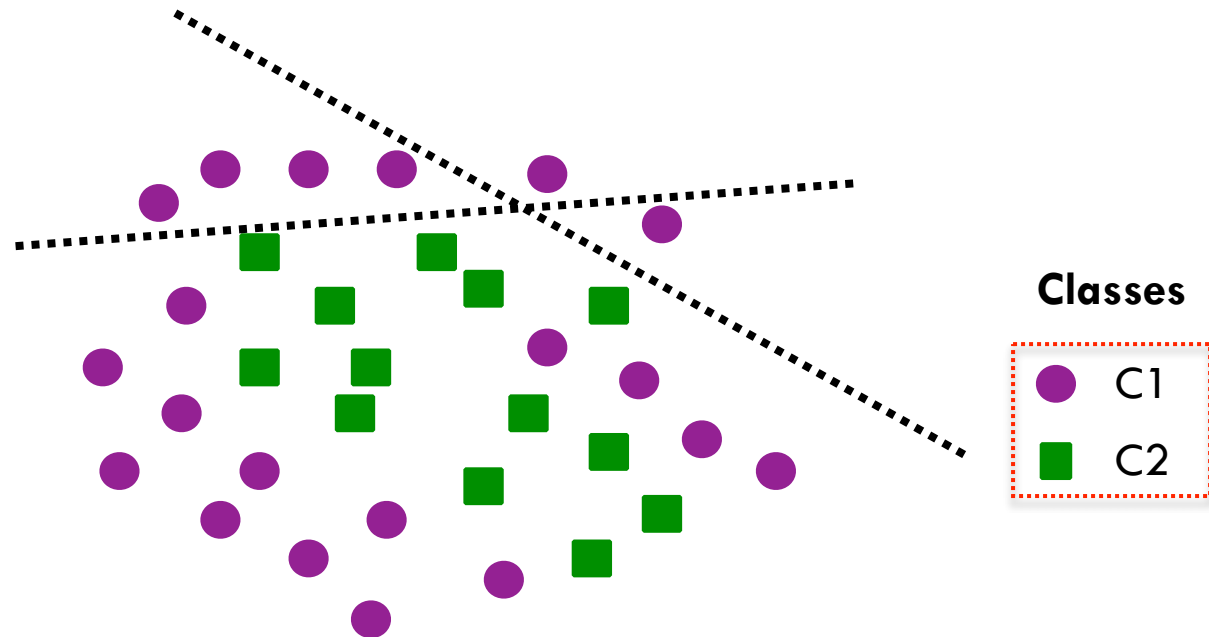
Exemplo



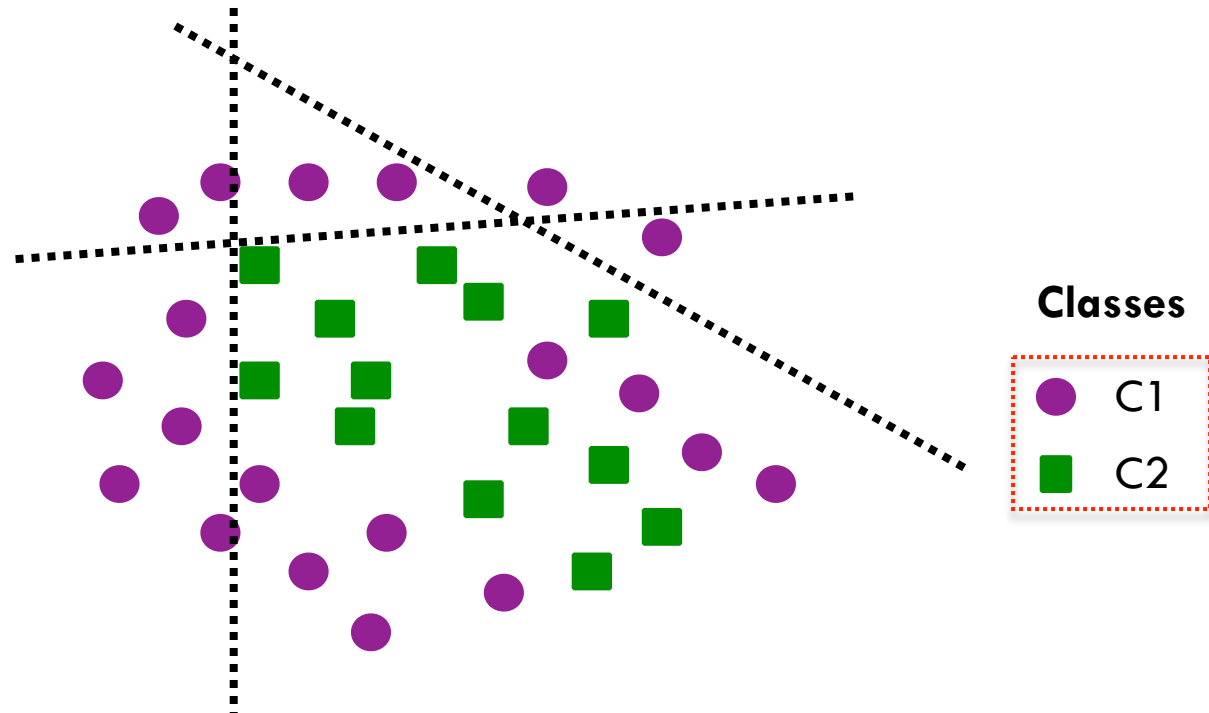
Exemplo



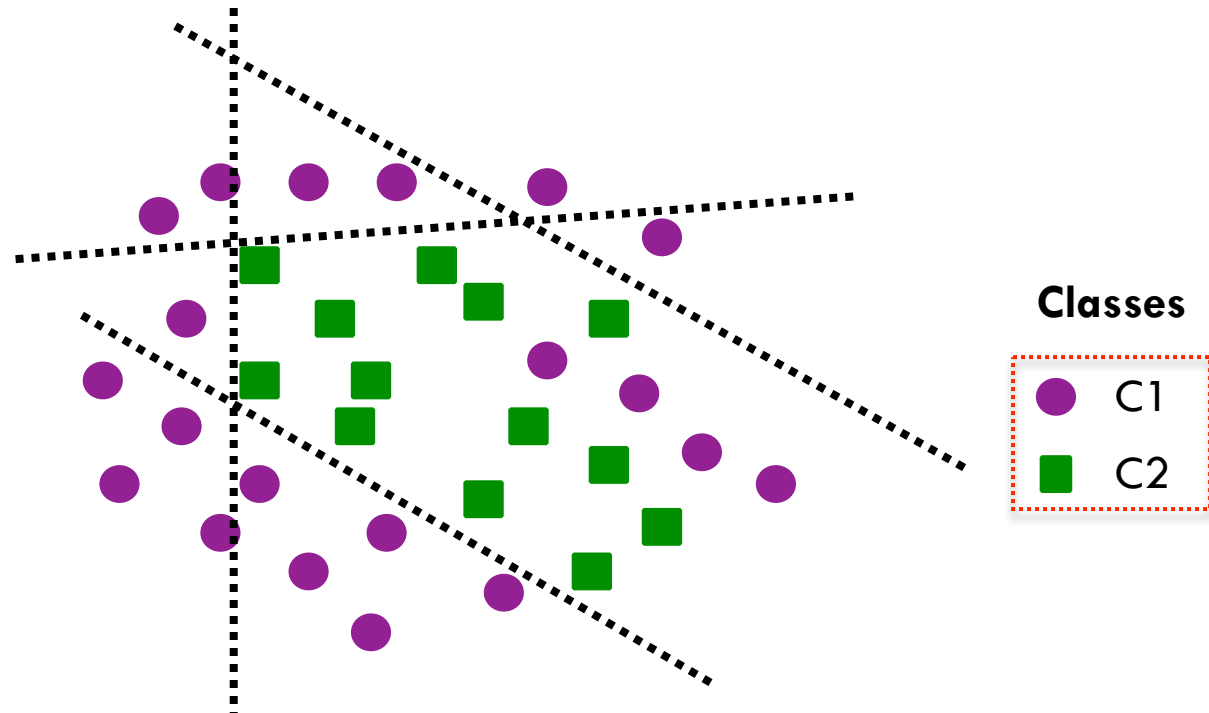
Exemplo



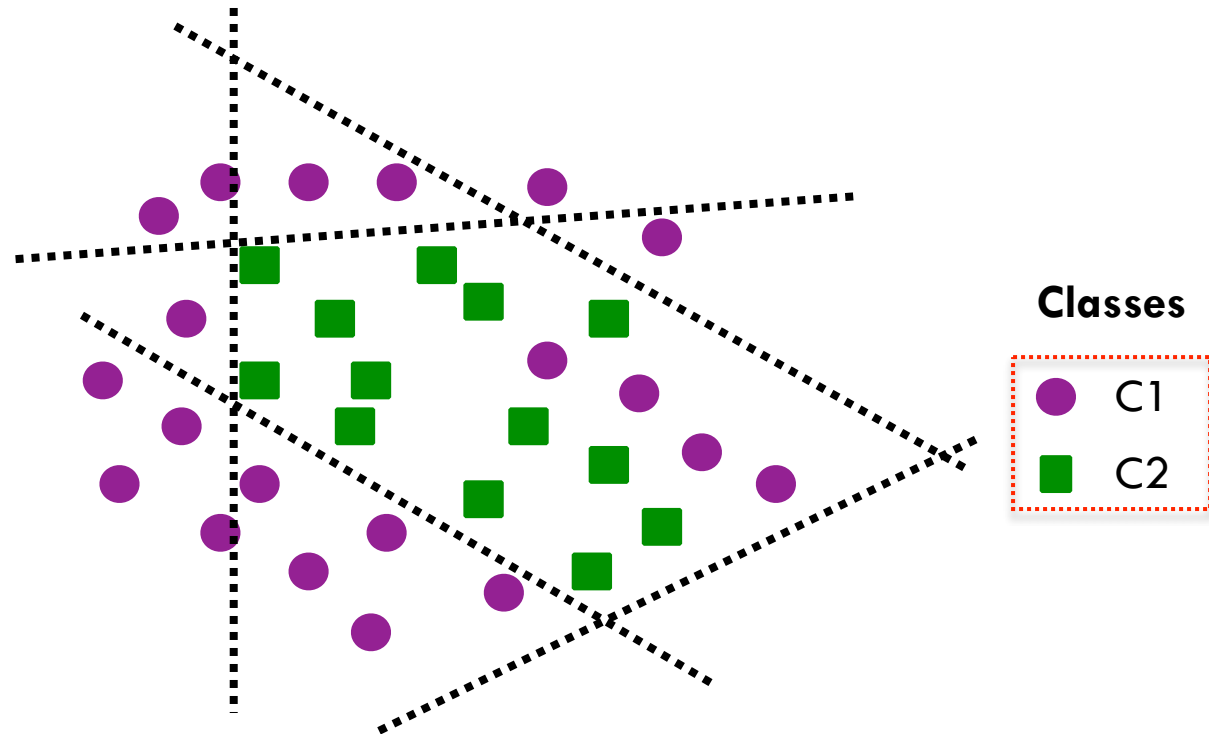
Exemplo



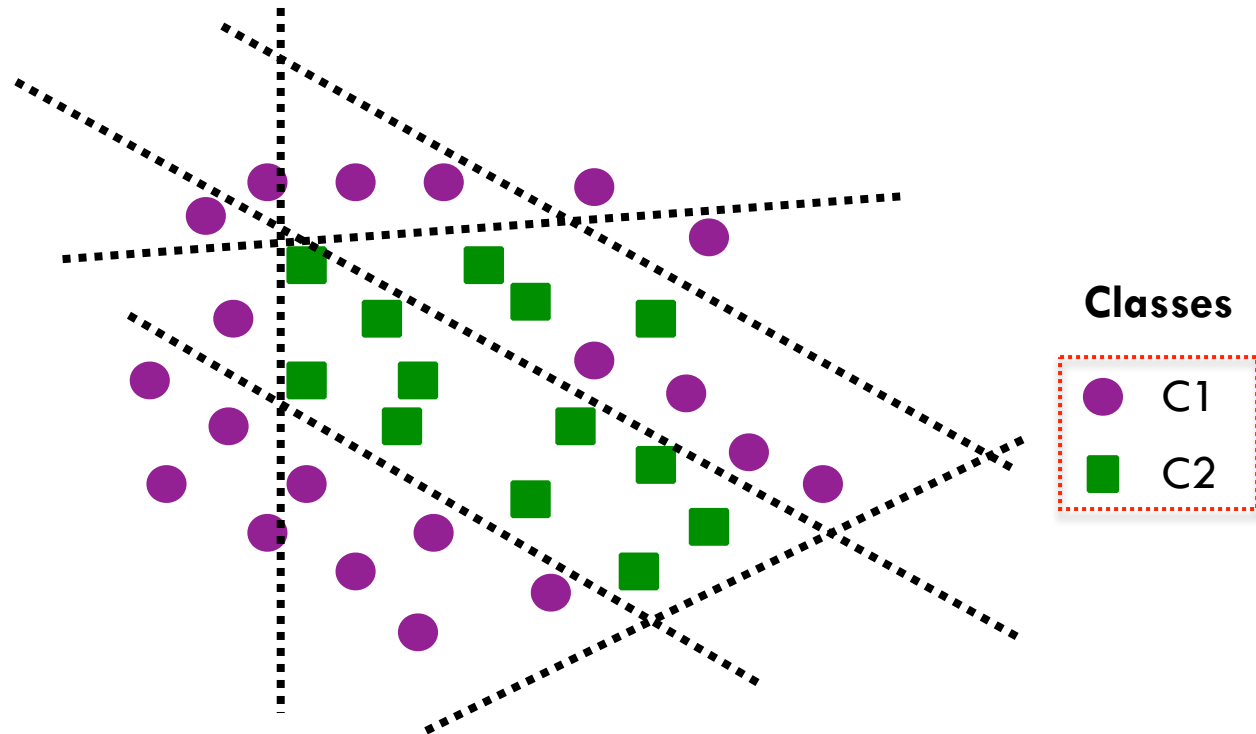
Exemplo



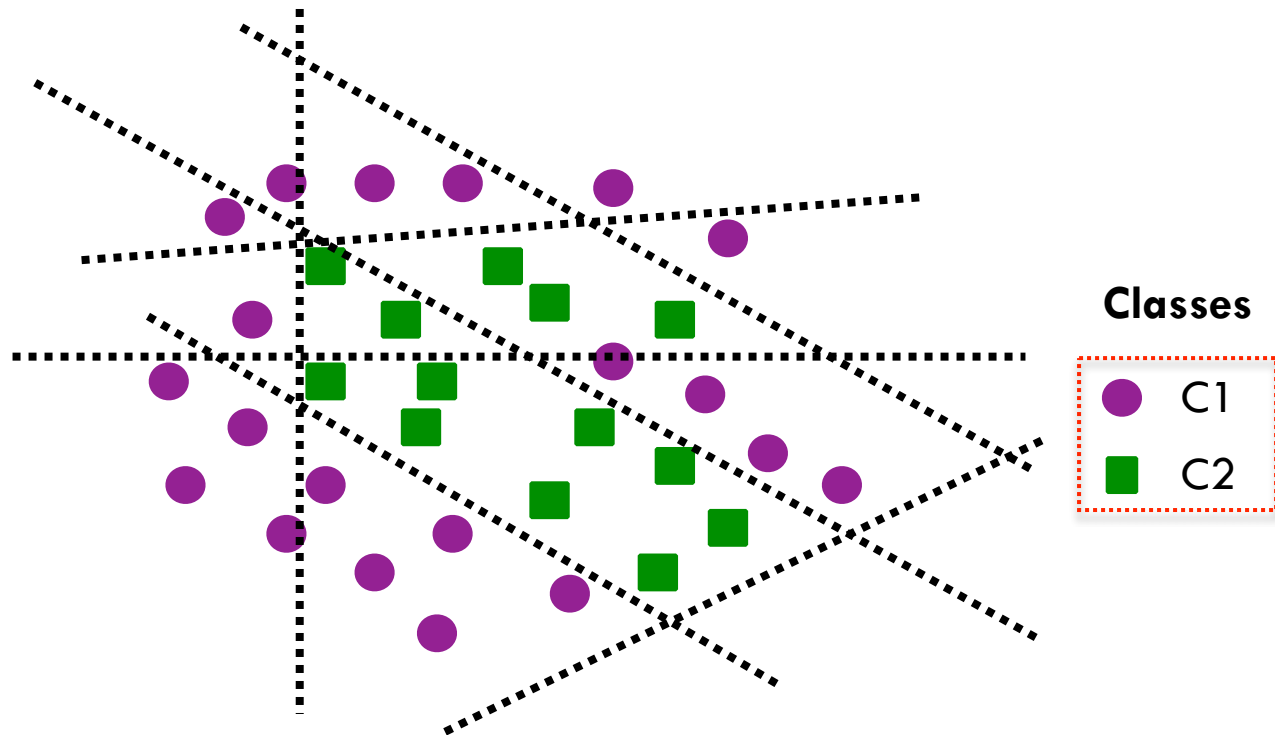
Exemplo



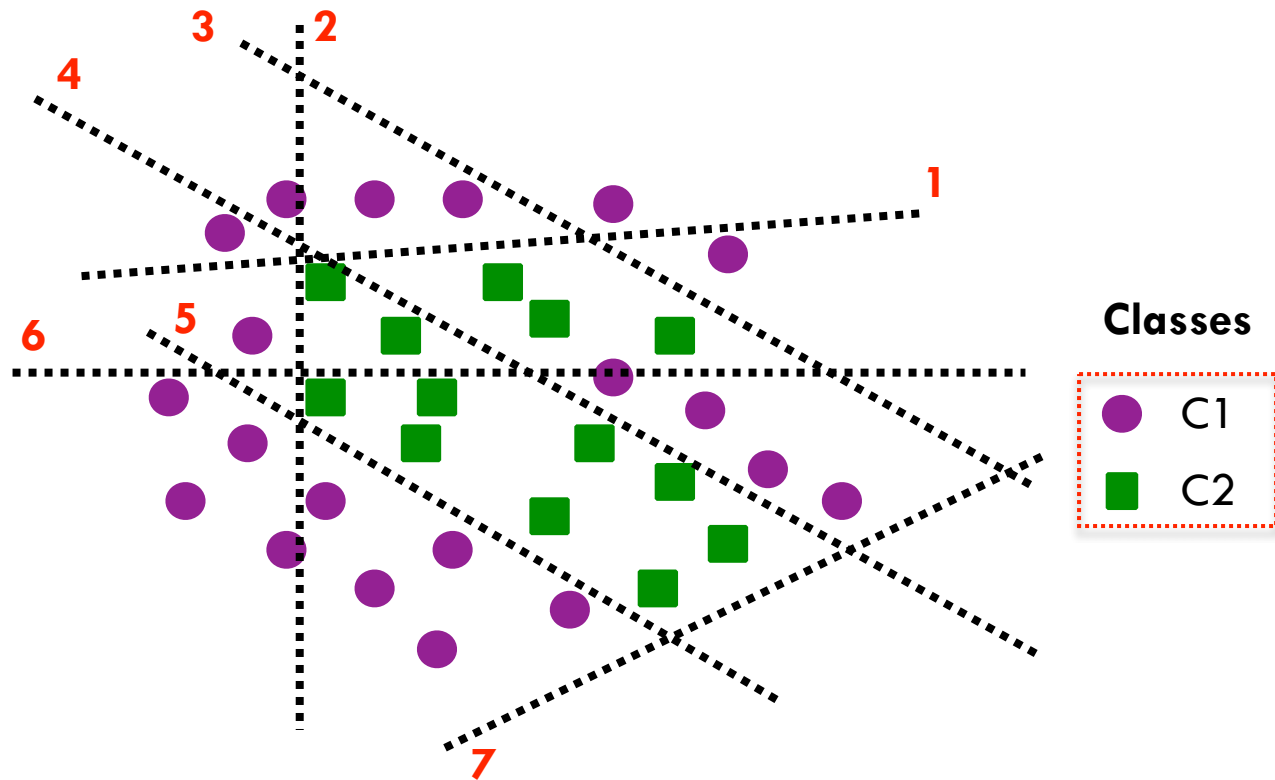
Exemplo



Exemplo



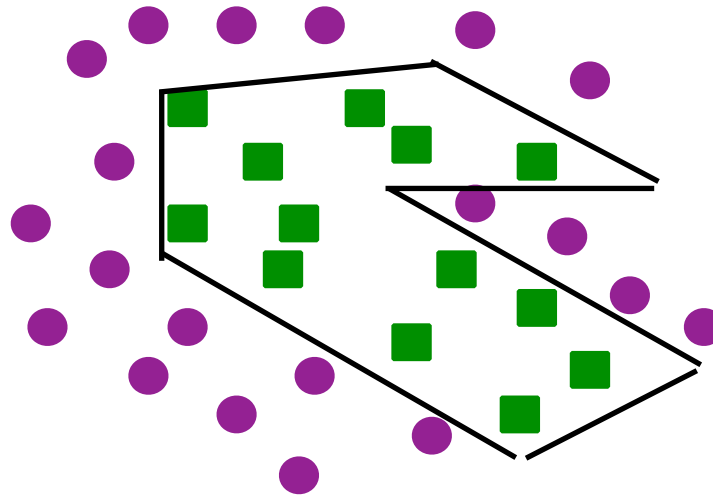
Exemplo



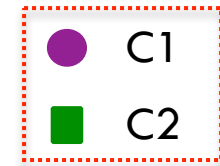
7 hiperplanos = 7 neurônios

Exemplo

No nosso exemplo hipotético, precisaríamos de uma região convexa com 7 hiperplanos para separar as classes corretamente.



Classes



7 hiperplanos = 1 região convexa

Roteiro

- 1 Introdução
- 2 Multilayer Perceptron
- 3 Formalização
- 4 Treinamento
- 5 Backpropagation
- 6 Referências

Formalização

- **MLP:**

- $\tau = [x(n), d(n)] \rightarrow$ exemplo de treinamento
- $y_j(n) \rightarrow$ sinal produzido na saída do neurônio j na camada de saída, estimulado por $x(n)$, aplicado na camada de entrada

Formalização

- **MLP:**

- $\tau = [x(n), d(n)] \rightarrow$ exemplo de treinamento
- $y_j(n) \rightarrow$ sinal produzido na saída do neurônio j na camada de saída, estimulado por $x(n)$, aplicado na camada de entrada
- Sinal do erro produzido pelo neurônio j é:

$$e_j(n) = d_j(n) - y_j(n)$$

Formalização

- O erro instantâneo produzido no neurônio j é dado por:

Formalização

- O erro instantâneo produzido no neurônio **j** é dado por:

$$\varepsilon_j(n) = \frac{1}{2}e_j^2(n)$$

Formalização

- O erro instantâneo produzido no neurônio **j** é dado por:

$$\varepsilon_j(n) = \frac{1}{2}e_j^2(n)$$

- Somando o erro de todos os neurônios da camada de saída:

Formalização

- O erro instantâneo produzido no neurônio **j** é dado por:

$$\varepsilon_j(n) = \frac{1}{2}e_j^2(n)$$

- Somando o erro de todos os neurônios da camada de saída:

$$\varepsilon(n) = \sum_{j \in C} \varepsilon_j(n)$$

Formalização

- O erro instantâneo produzido no neurônio j é dado por:

$$\varepsilon_j(n) = \frac{1}{2}e_j^2(n)$$

- Somando o erro de todos os neurônios da camada de saída:

$$\varepsilon(n) = \sum_{j \in C} \varepsilon_j(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n)$$

Formalização

- O erro instantâneo produzido no neurônio **j** é dado por:

$$\varepsilon_j(n) = \frac{1}{2}e_j^2(n)$$

- Somando o erro de todos os neurônios da camada de saída:

$$\varepsilon(n) = \sum_{j \in C} \varepsilon_j(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n)$$

C é o conjunto de neurônios de saída

Formalização

- Se houverem **N** exemplos de treinamento, o erro médio sobre todos os exemplos (risco empírico) é dado por:

Formalização

- Se houverem **N** exemplos de treinamento, o erro médio sobre todos os exemplos (risco empírico) é dado por:

$$\varepsilon_{avg}(N) =$$

Formalização

- Se houverem **N** exemplos de treinamento, o erro médio sobre todos os exemplos (risco empírico) é dado por:

$$\varepsilon_{avg}(N) = \frac{1}{N} \sum_{n=1}^N \varepsilon(n)$$

Formalização

- Se houverem **N** exemplos de treinamento, o erro médio sobre todos os exemplos (risco empírico) é dado por:

$$\varepsilon_{avg}(N) = \frac{1}{N} \sum_{n=1}^N \varepsilon(n) = \frac{1}{2N} \sum_{n=1}^N \sum_{j \in C} e_j^2(n)$$

Formalização

- Se houverem **N** exemplos de treinamento, o erro médio sobre todos os exemplos (risco empírico) é dado por:

$$\varepsilon_{avg}(N) = \frac{1}{N} \sum_{n=1}^N \varepsilon(n) = \frac{1}{2N} \sum_{n=1}^N \sum_{j \in C} e_j^2(n)$$

Erro quadrático médio da época considera todos os neurônios da camada de saída **C** e todos os exemplos do conjunto de treinamento (**N**)

Roteiro

- 1 Introdução
- 2 Multilayer Perceptron
- 3 Formalização
- 4 Treinamento
- 5 Backpropagation
- 6 Referências

Treinamento



Treinamento



1

Treinamento Online

2

Treinamento em Batch

Treinamento



1

Treinamento Online

Treinamento

1

Treinamento Online



Ajuste dos pesos (W) ocorre após a apresentação de cada exemplo



A busca no espaço multidimensional de pesos torna-se estocástica

Treinamento

1

Treinamento Online



Ajuste dos pesos (W) ocorre após a apresentação de cada exemplo



A busca no espaço multidimensional de pesos torna-se estocástica



Menos suscetível a ficar preso em mínimos locais



Quando há redundância, tira vantagem ao ajustar os pesos

Treinamento

1

Treinamento Online



Ajuste dos pesos (W) ocorre após a apresentação de cada exemplo



A busca no espaço multidimensional de pesos torna-se estocástica



Menos suscetível a ficar preso em mínimos locais



Quando há redundância, tira vantagem ao ajustar os pesos



Detecta melhor pequenas mudanças nos dados de treinamento



Simple de implementar / Bons resultados em problemas difíceis

Treinamento



2

Treinamento em Batch

Treinamento

2

Treinamento em Batch



Apresenta todos os exemplos para a rede



Só depois faz o ajuste de pesos. 1 ajuste para uma época completa

Treinamento

2

Treinamento em Batch



Apresenta todos os exemplos para a rede



Só depois faz o ajuste de pesos. 1 ajuste para uma época completa



Estimativa mais precisa do vetor de gradientes



Paralelização do processo de aprendizado

Treinamento

2

Treinamento em Batch



Apresenta todos os exemplos para a rede



Só depois faz o ajuste de pesos. 1 ajuste para uma época completa



Estimativa mais precisa do vetor de gradientes



Paralelização do processo de aprendizado



Mais difícil de detectar mudanças pequenas nos dados



Mais se há exemplos redundantes, não consegue identificar, pois ajusta os pesos para todos os exemplos de uma única vez

Treinamento

Em ambos os casos

- Vários experimentos, iniciando W com valores diferentes
- desempenho da rede é a média dos diferentes experimentos
- Análise da Curva de Aprendizado
- erro da época x época

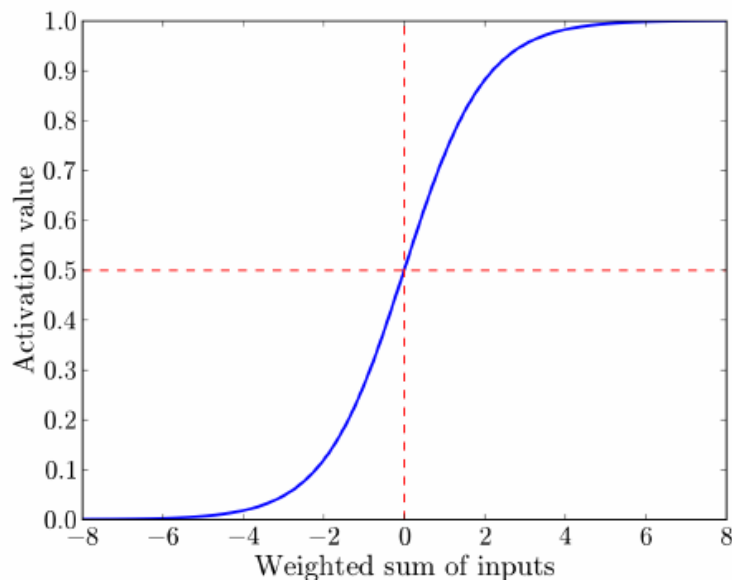
Funções de Ativação

- ϕ deve ser diferenciável:

Funções de Ativação

- **ϕ deve ser diferenciável:**
 - funções sigmoidal / logística

$$\varphi_j(v_j(n)) = \frac{1}{1 + \exp(-av_j(n))}, \quad a > 0$$

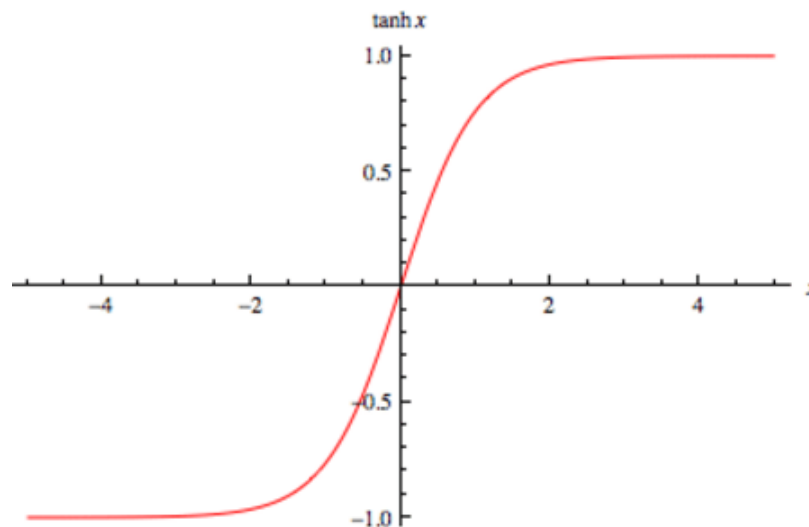


Funções de Ativação

- função tangente hiperbólica:

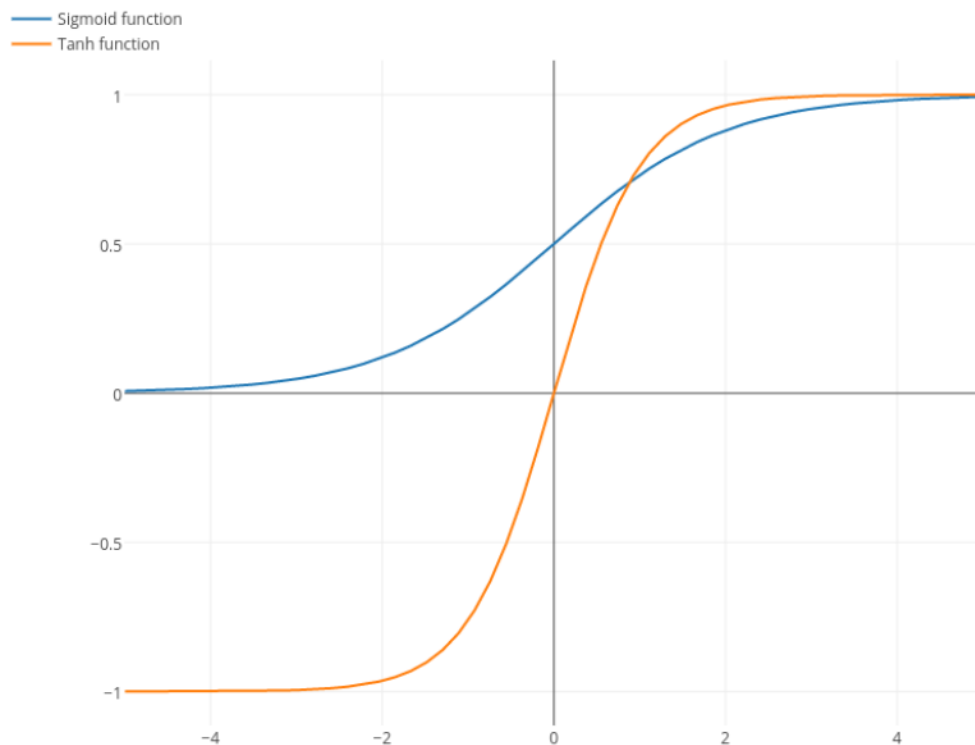
$$\varphi_j(v_j(n)) = a \tanh(bv_j(n))$$

- a e b são constantes positivas
- amplitude do sinal de saída: $-a \leq y_i \leq +a$



Funções de Ativação

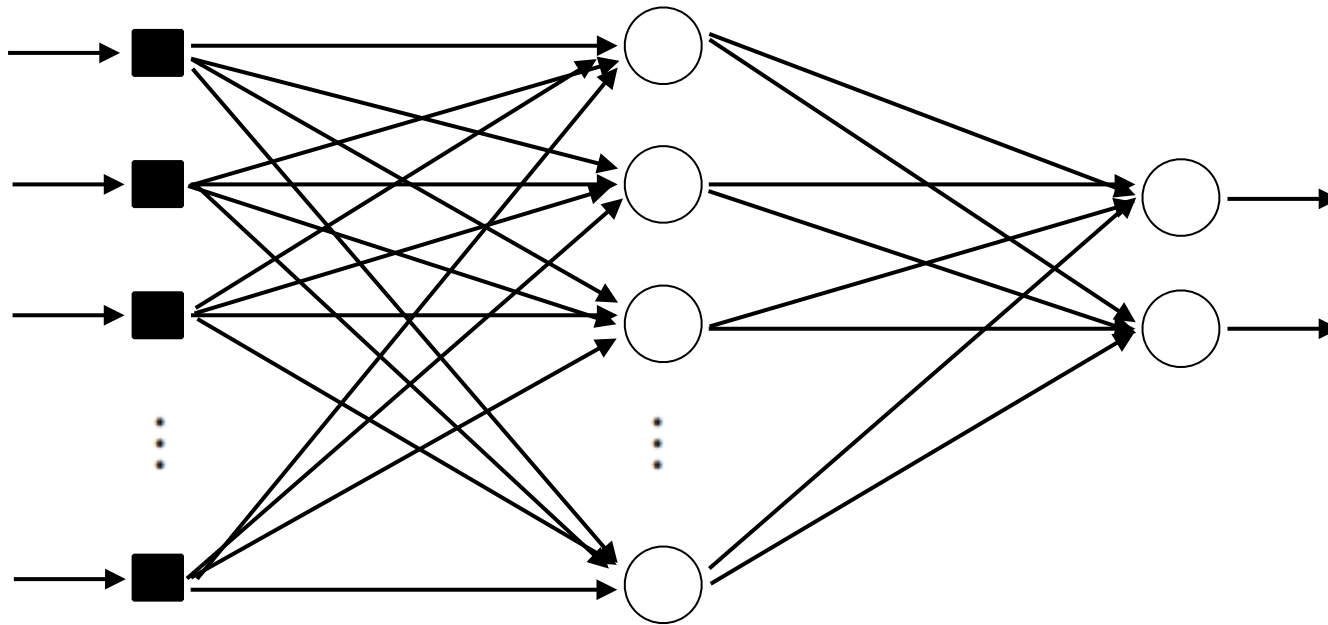
- Comparativo entre as duas formas de funções de ativação
 - $\tanh$ x sigmoid



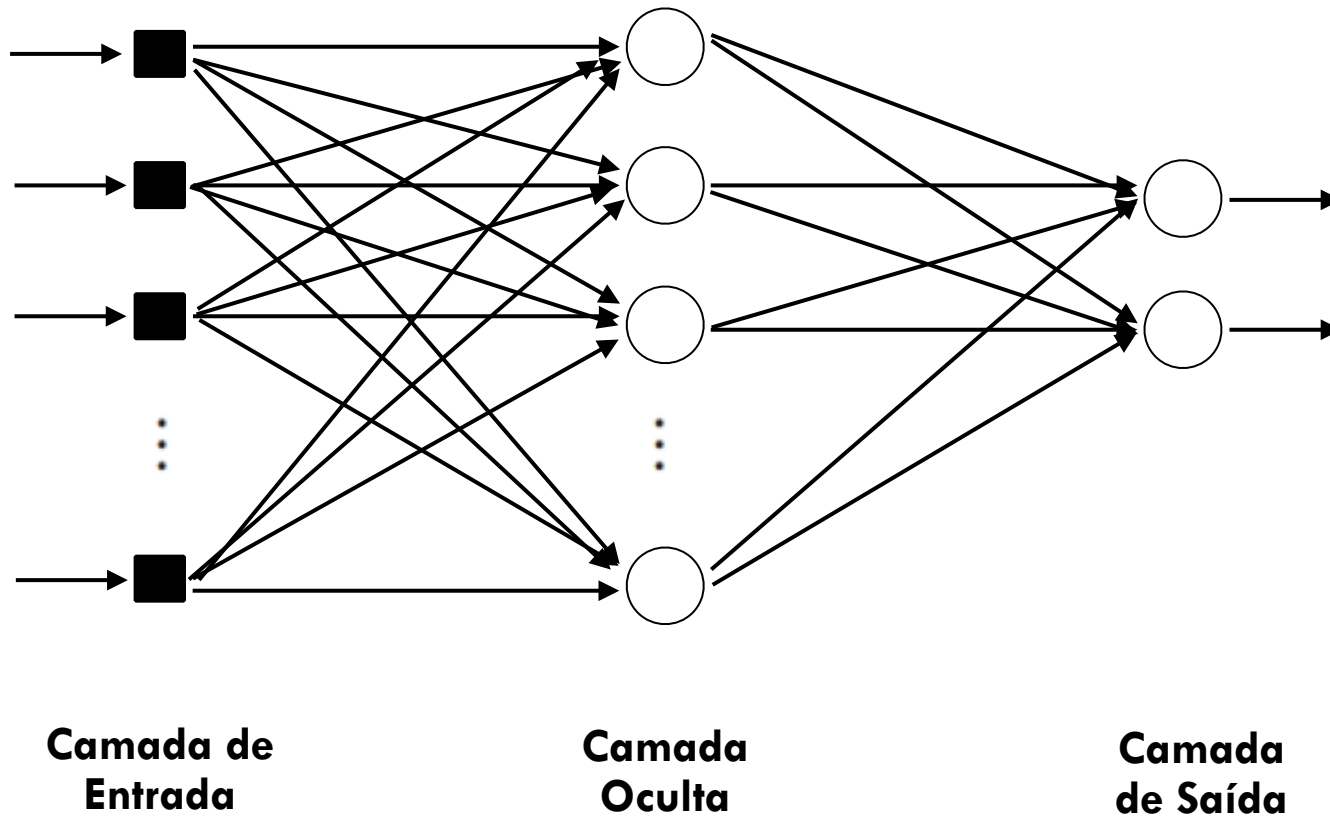
Roteiro

- 1 Introdução
- 2 Multilayer Perceptron
- 3 Formalização
- 4 Treinamento
- 5 Backpropagation
- 6 Referências

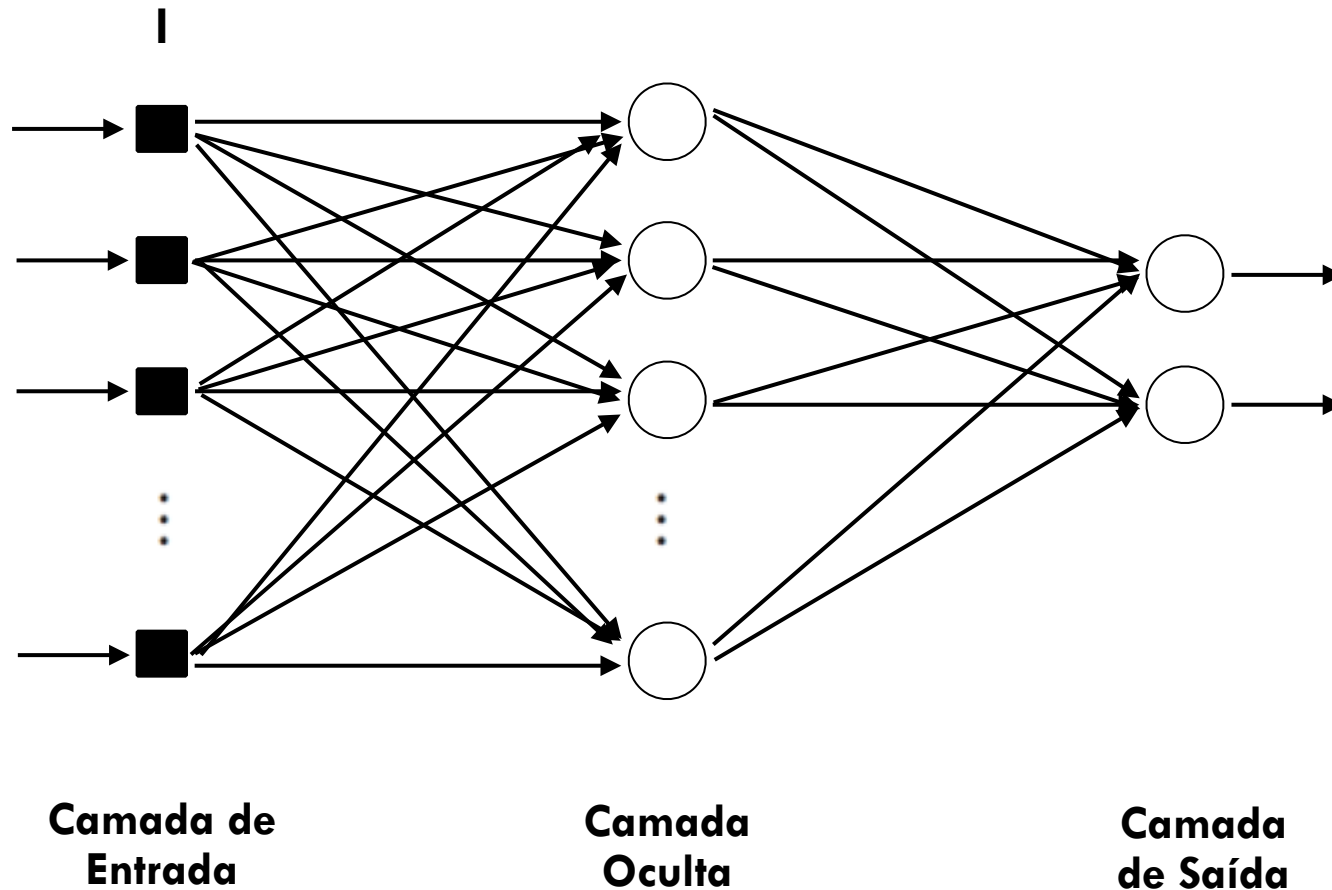
Backpropagation



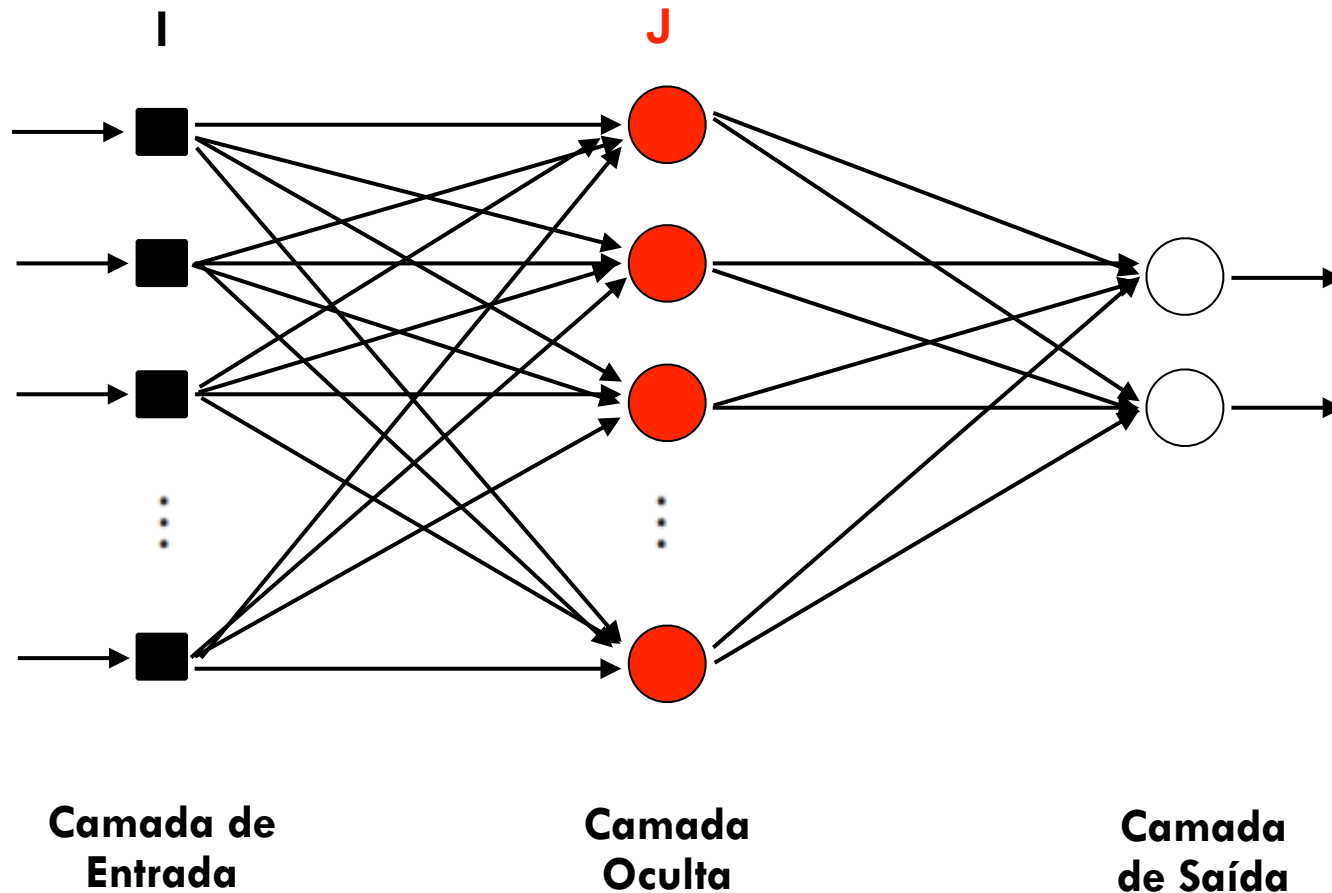
Backpropagation



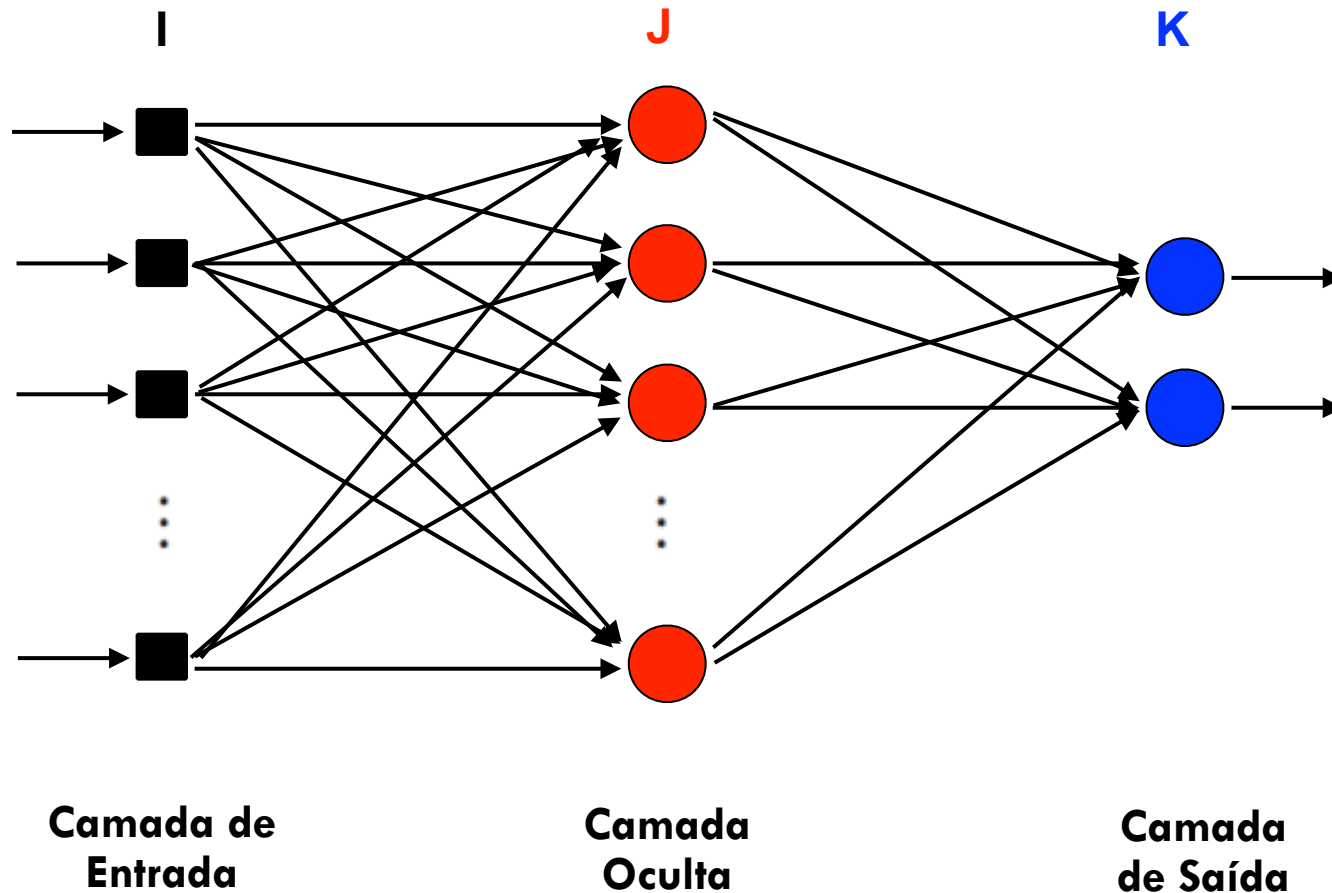
Backpropagation



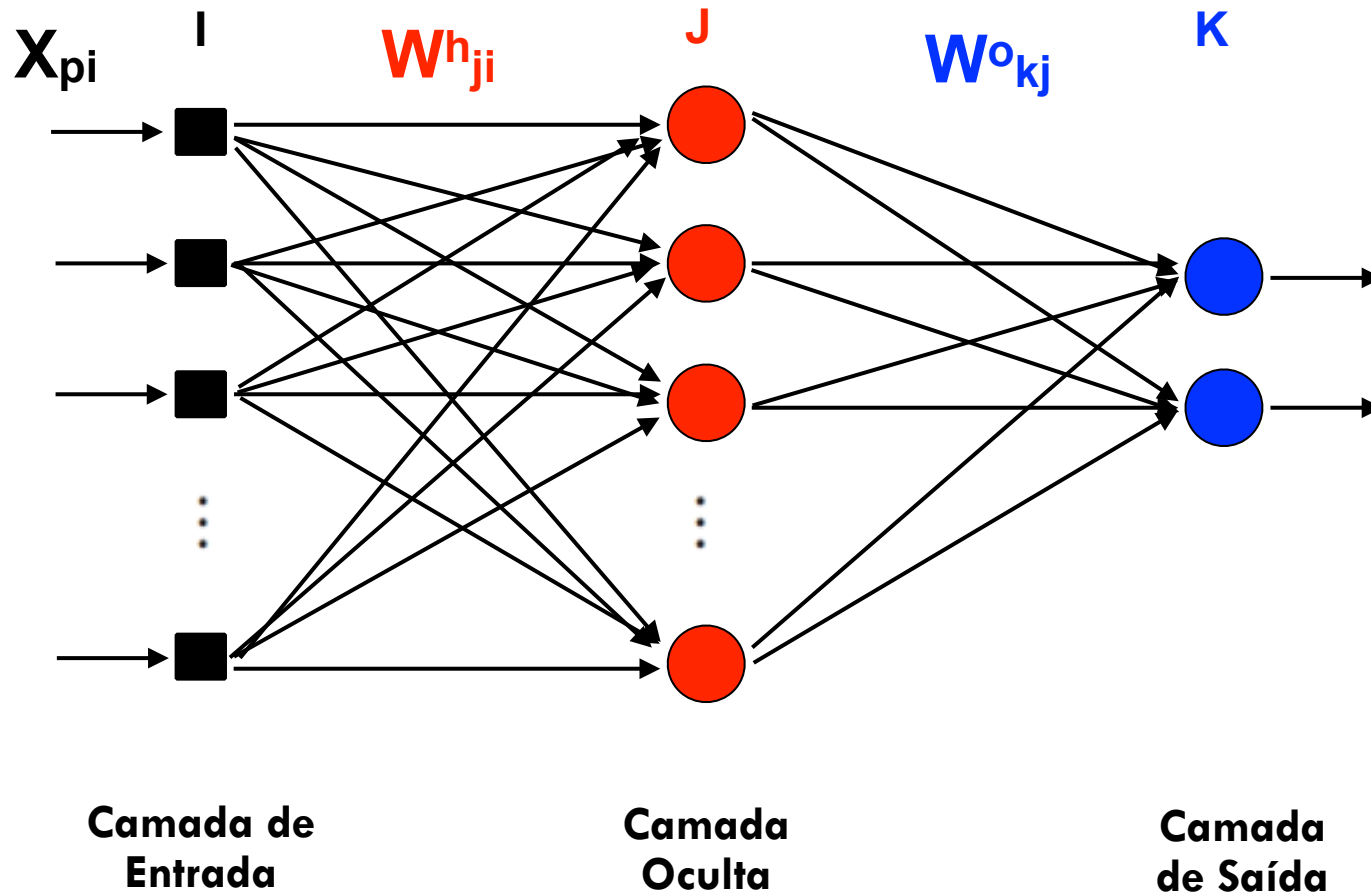
Backpropagation



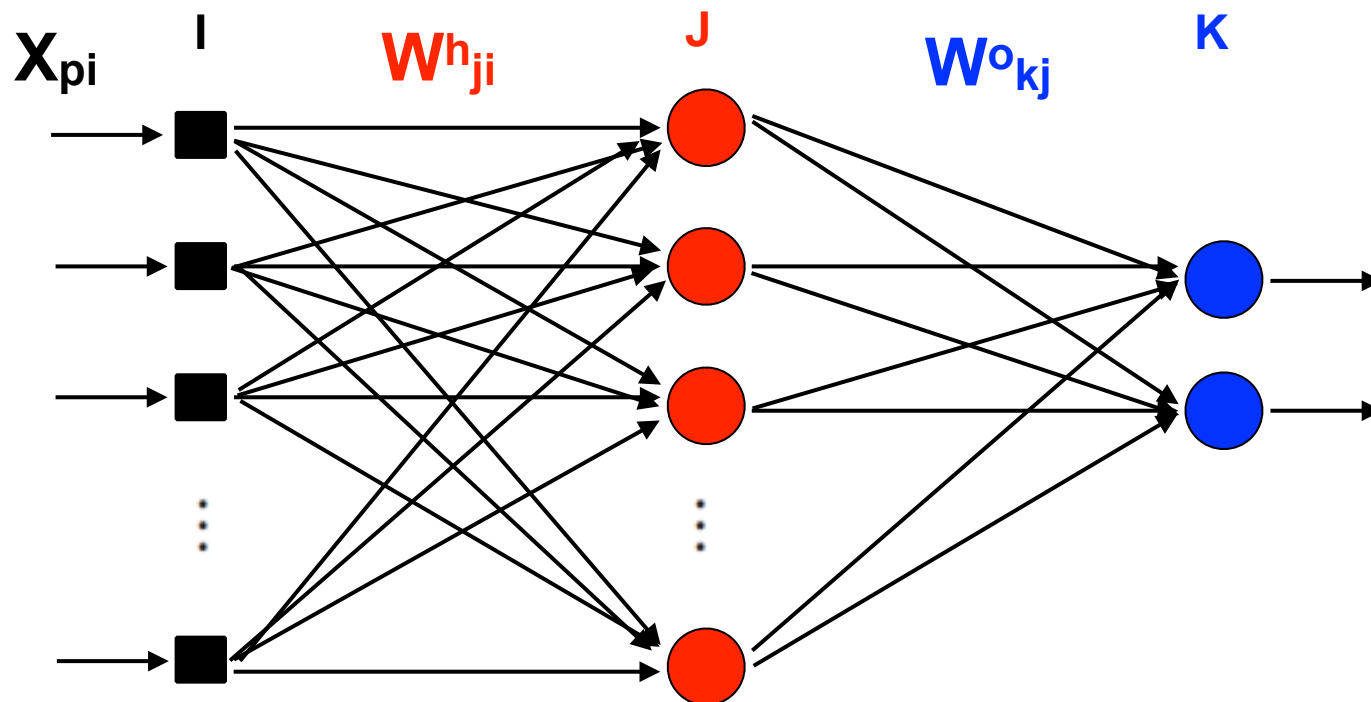
Backpropagation



Backpropagation



Backpropagation

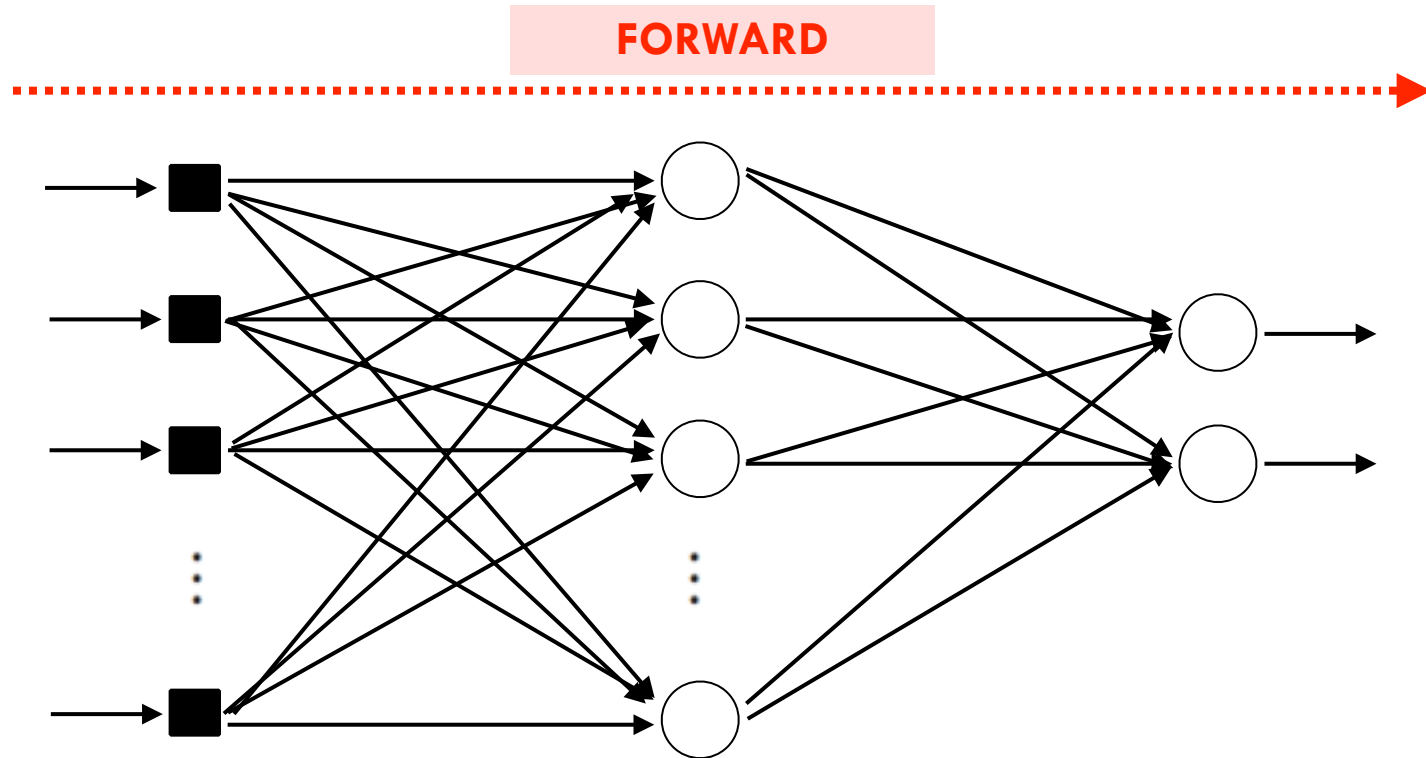


Os pesos sinápticos são matrizes:

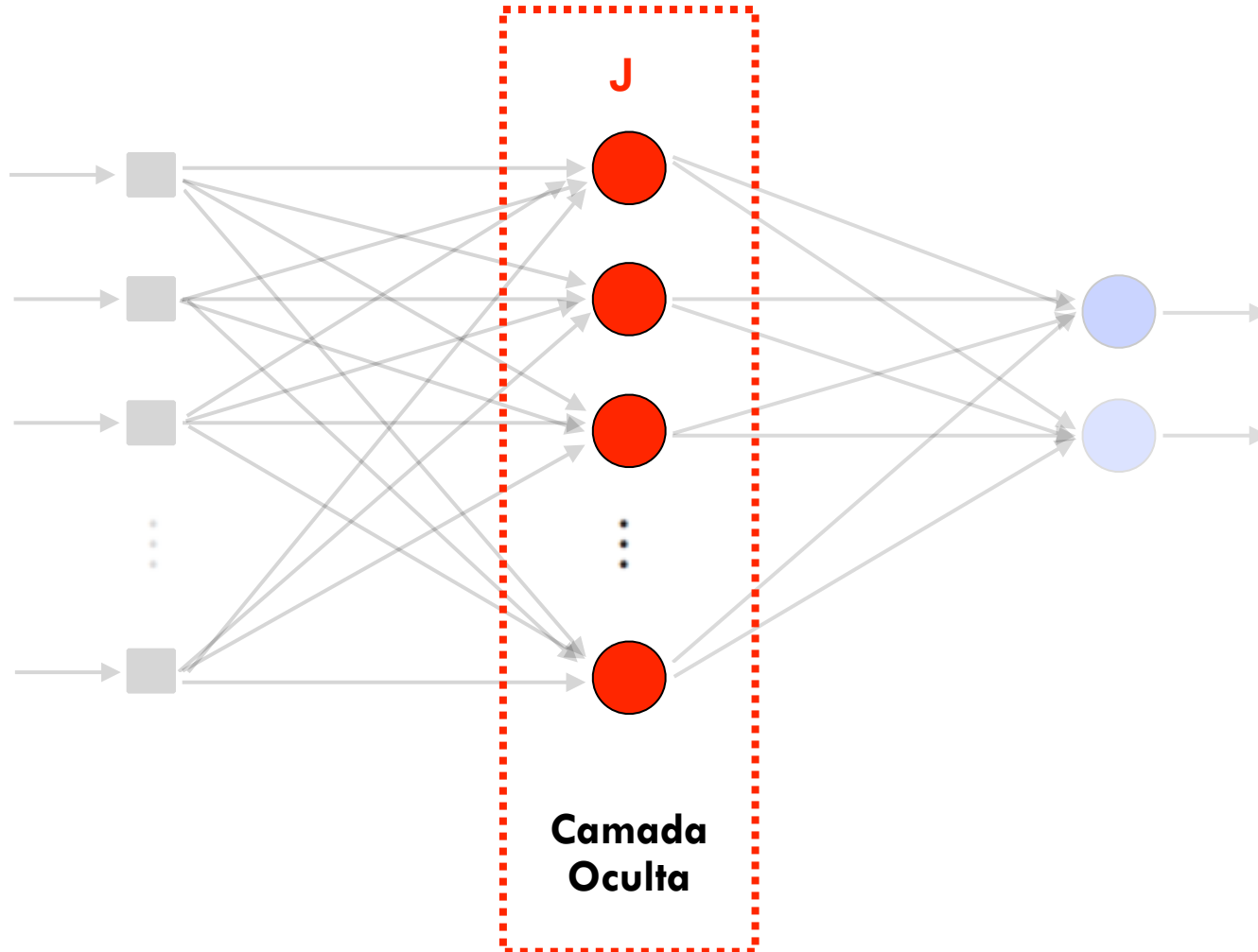
W^h - conecta camada **oculta** e a camada de **entrada**

W^o - conecta a camada de **saída** e a camada **oculta**

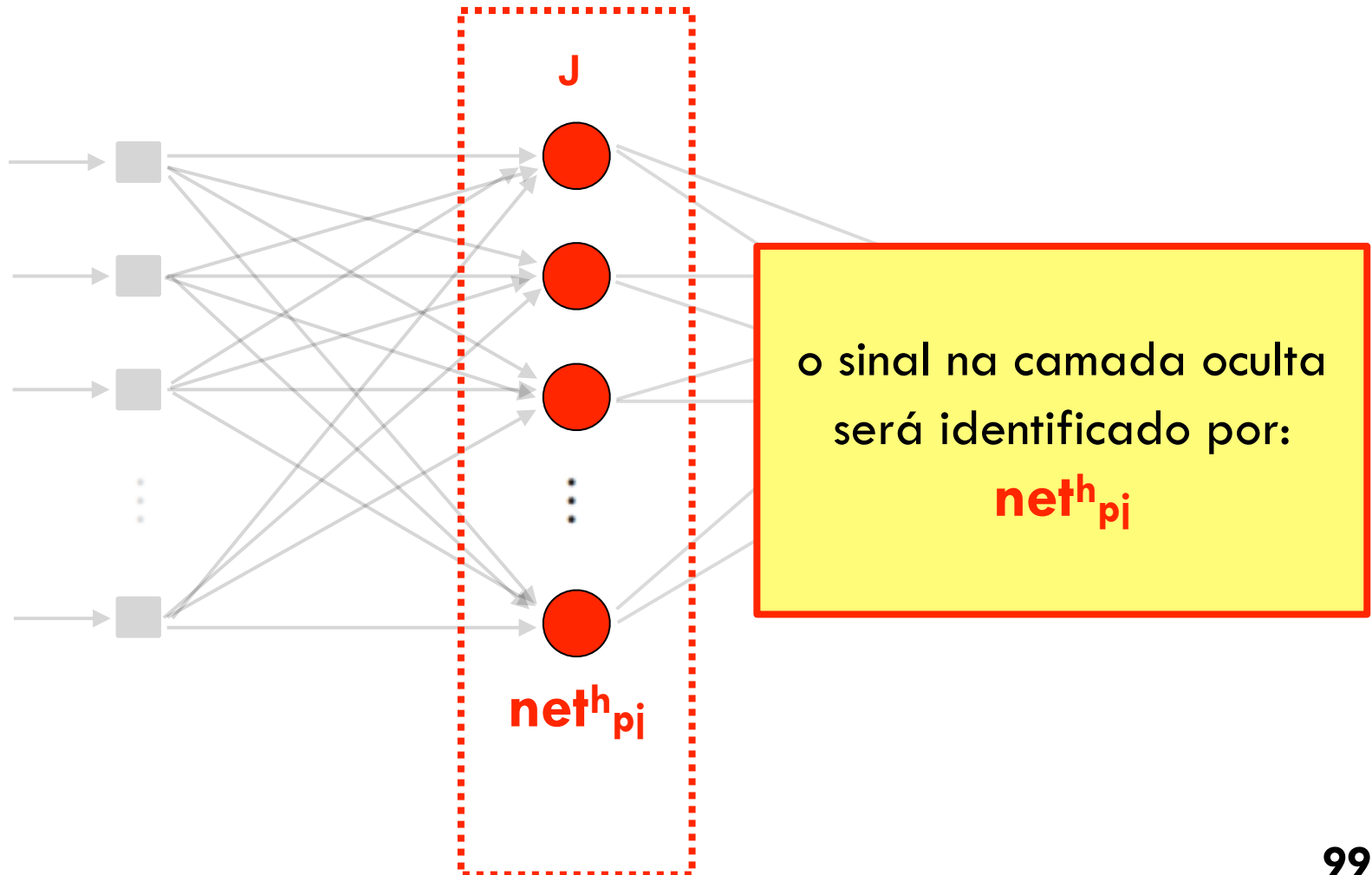
Backpropagation



Backpropagation



Backpropagation



Backpropagation

- 1 Sinal no neurônio de índice **J** na camada escondida:

$$net_{pj}^h = \sum_{i=1}^N W_{ji}^h x_{pi} + \theta_j^h$$

Backpropagation

- 1 Sinal no neurônio de índice **J** na camada escondida:

$$net_{pj}^h = \sum_{i=1}^N W_{ji}^h x_{pi} + \theta_j^h$$

N : número de neurônios na camada de entrada

x_{pi} : exemplo de entrada fornecido para a rede

W_{ji}^h : pesos sinápticos que conectam os valores de entrada i ao neurônio J

θ_j^h : bias do neurônio J

Backpropagation

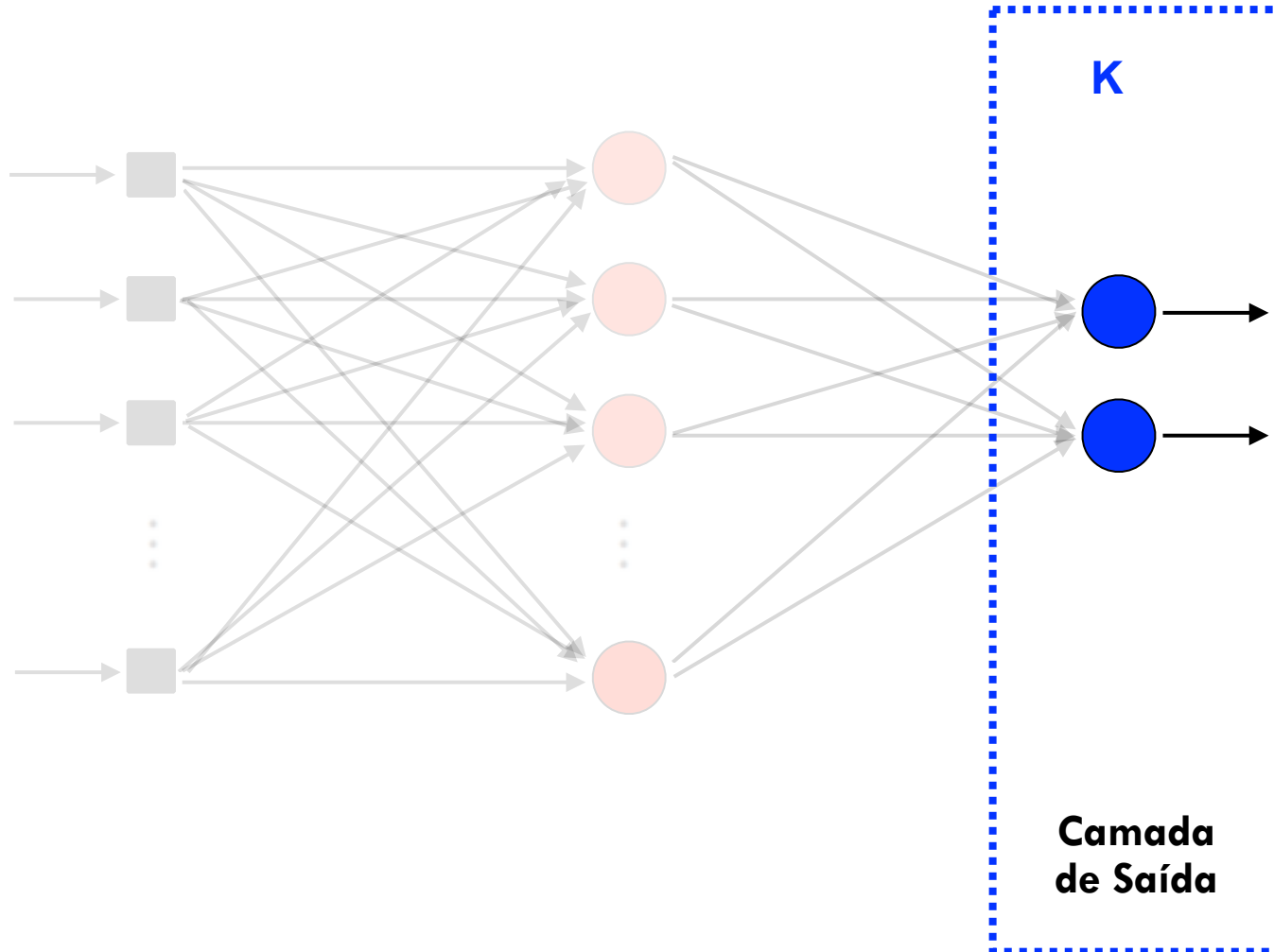
- 1 Sinal no neurônio de índice **J** na camada escondida:

$$net_{pj}^h = \sum_{i=1}^N W_{ji}^h x_{pi} + \theta_j^h$$

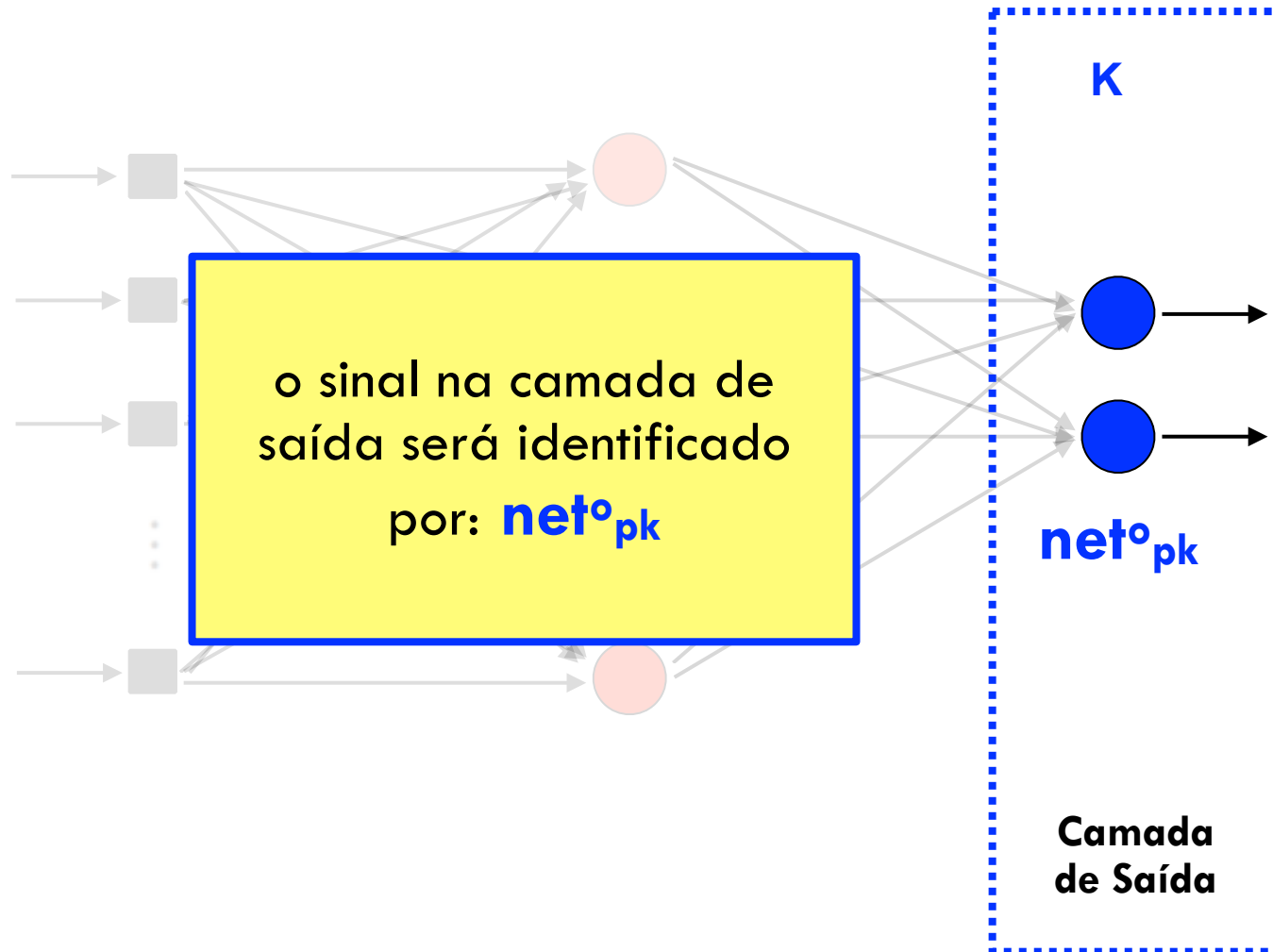
- 2 Ativação desse neurônio é igual a:

$$i_{pj} = f_j^h(net_{pj}^h)$$

Backpropagation



Backpropagation



Backpropagation

3 Sinal no neurônio de índice K na camada de saída:

Backpropagation

- 3 Sinal no neurônio de índice k na camada de saída:

$$net_{pk}^o = \sum_{j=1}^L W_{kj}^o i_{pj} + \theta_k^o$$

L : número de neurônios na camada oculta

i_{pj} : valor da ativação do neurônio i na camada oculta

W_{kj}^o : pesos sinápticos que conectam o neurônio J com a camada escondida

θ_k^o : bias do neurônio k

Backpropagation

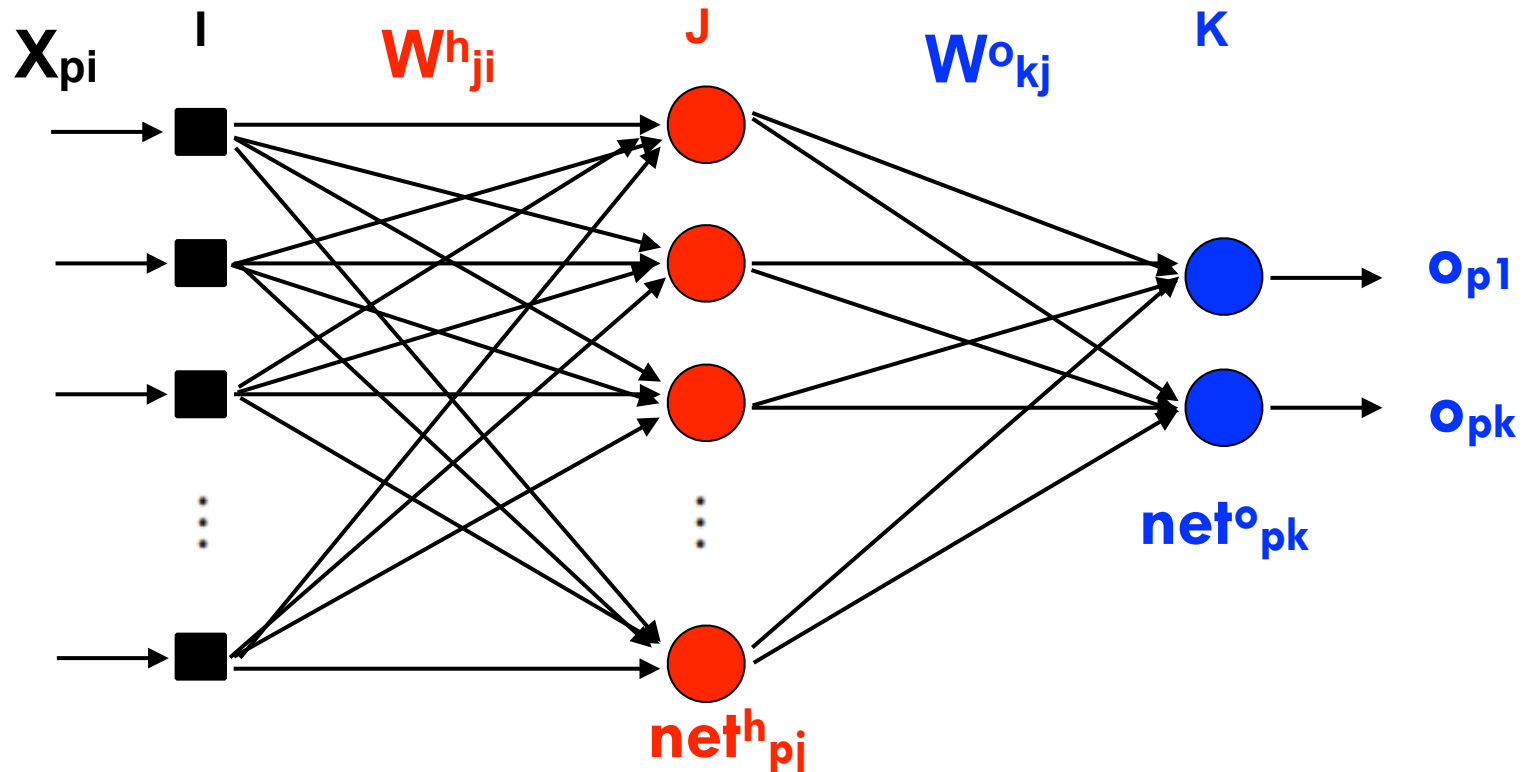
- 3 Sinal no neurônio de índice k na camada de saída:

$$net_{pk}^o = \sum_{j=1}^L W_{kj}^o i_{pj} + \theta_k^o$$

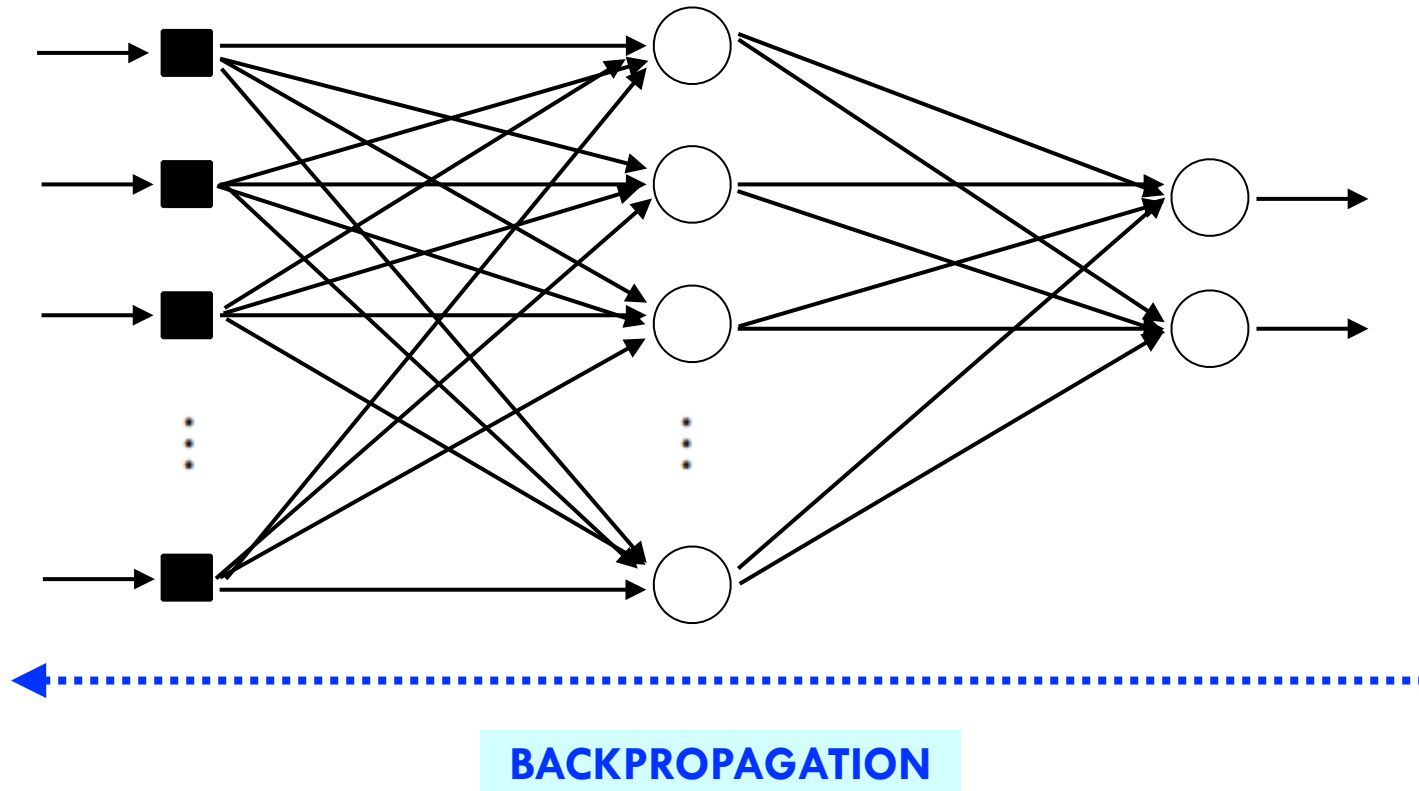
- 4 Saída desse neurônio é igual a:

$$o_{pk} = f_k^o(net_{pk}^o)$$

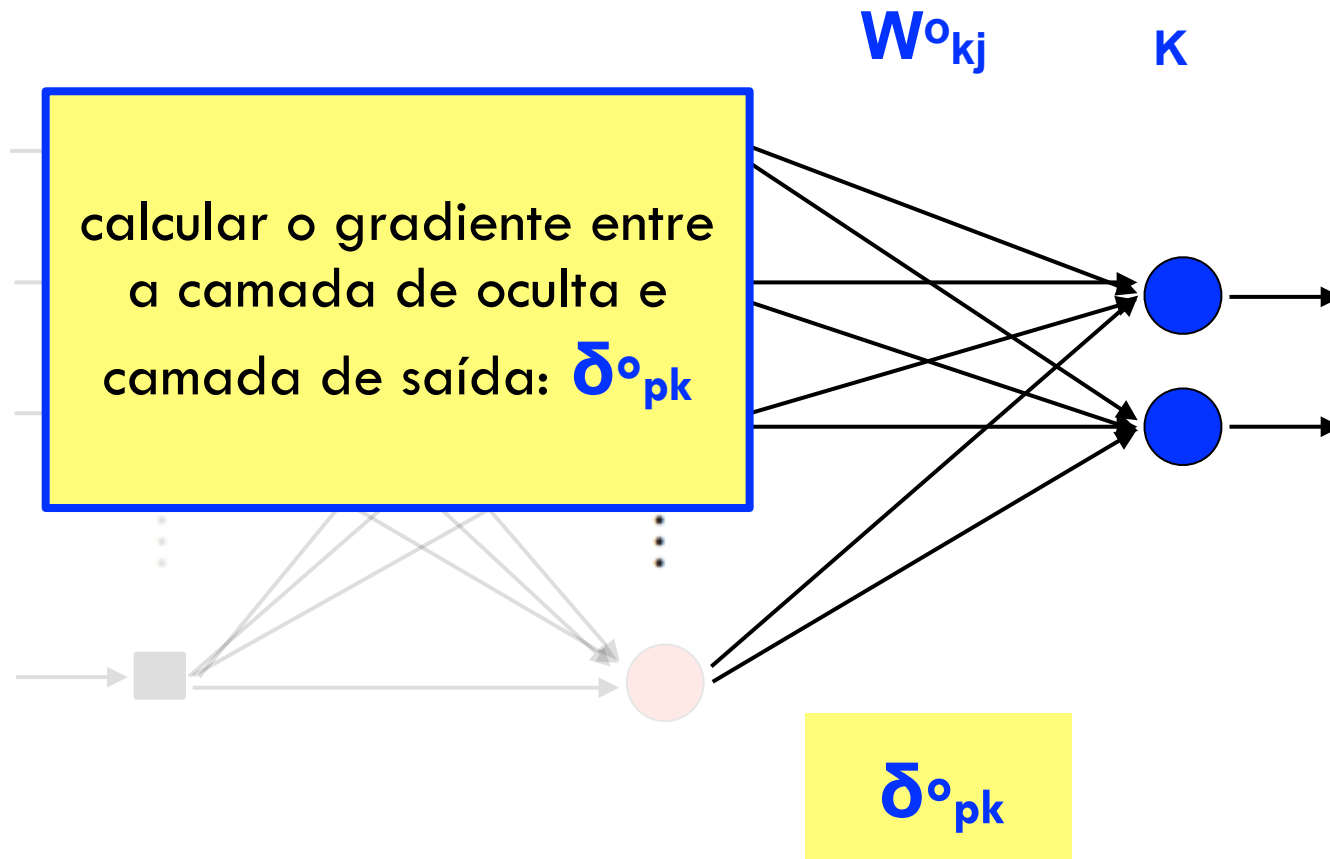
Backpropagation



Backpropagation



Backpropagation



Backpropagation

- 5 Calcular os termos de erro para as unidades de saída:

$$\delta_{pk}^o = (y_{pk} - o_{pk}) * f_k^o(net_{pk}^o)$$

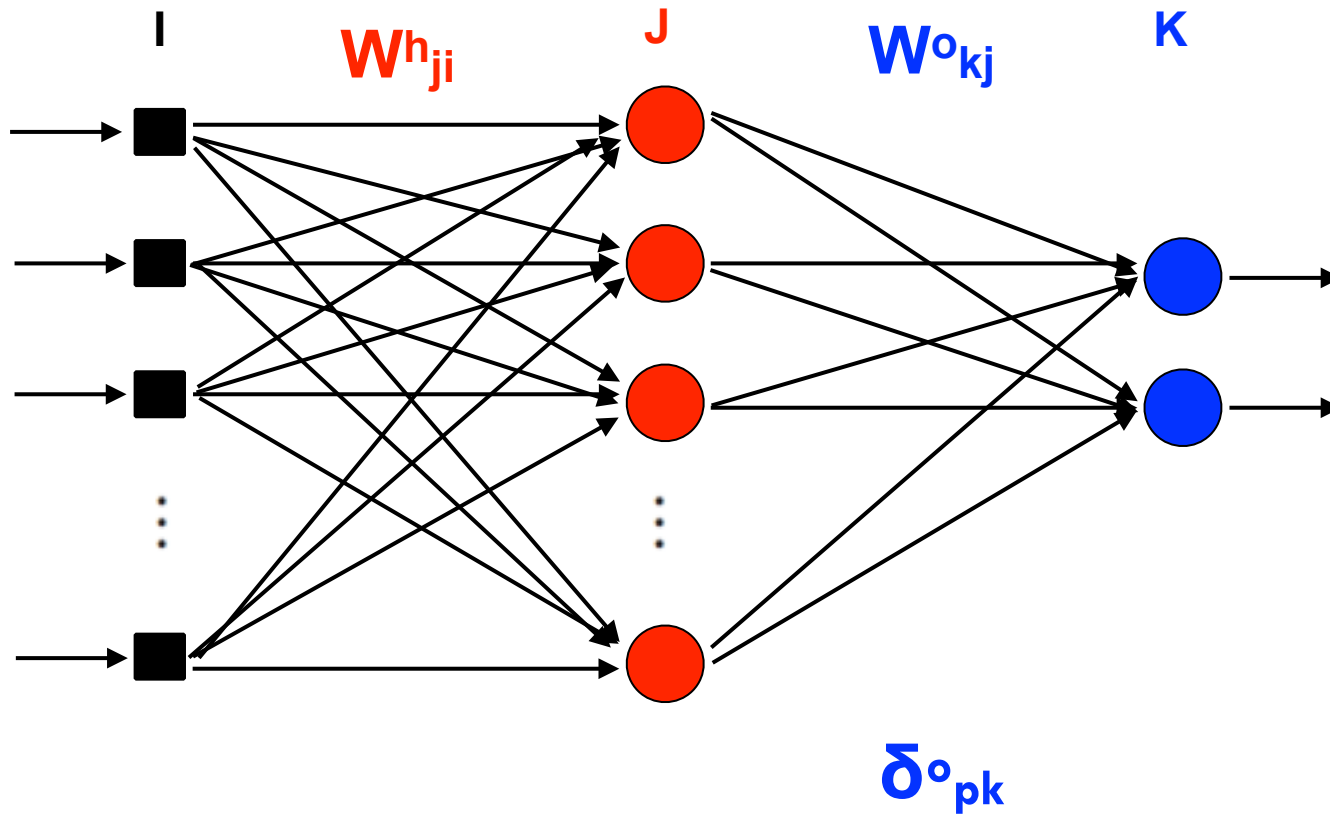
y_{pk} : saída esperada no neurônio k

o_{pk} : saída obtida no neurônio k

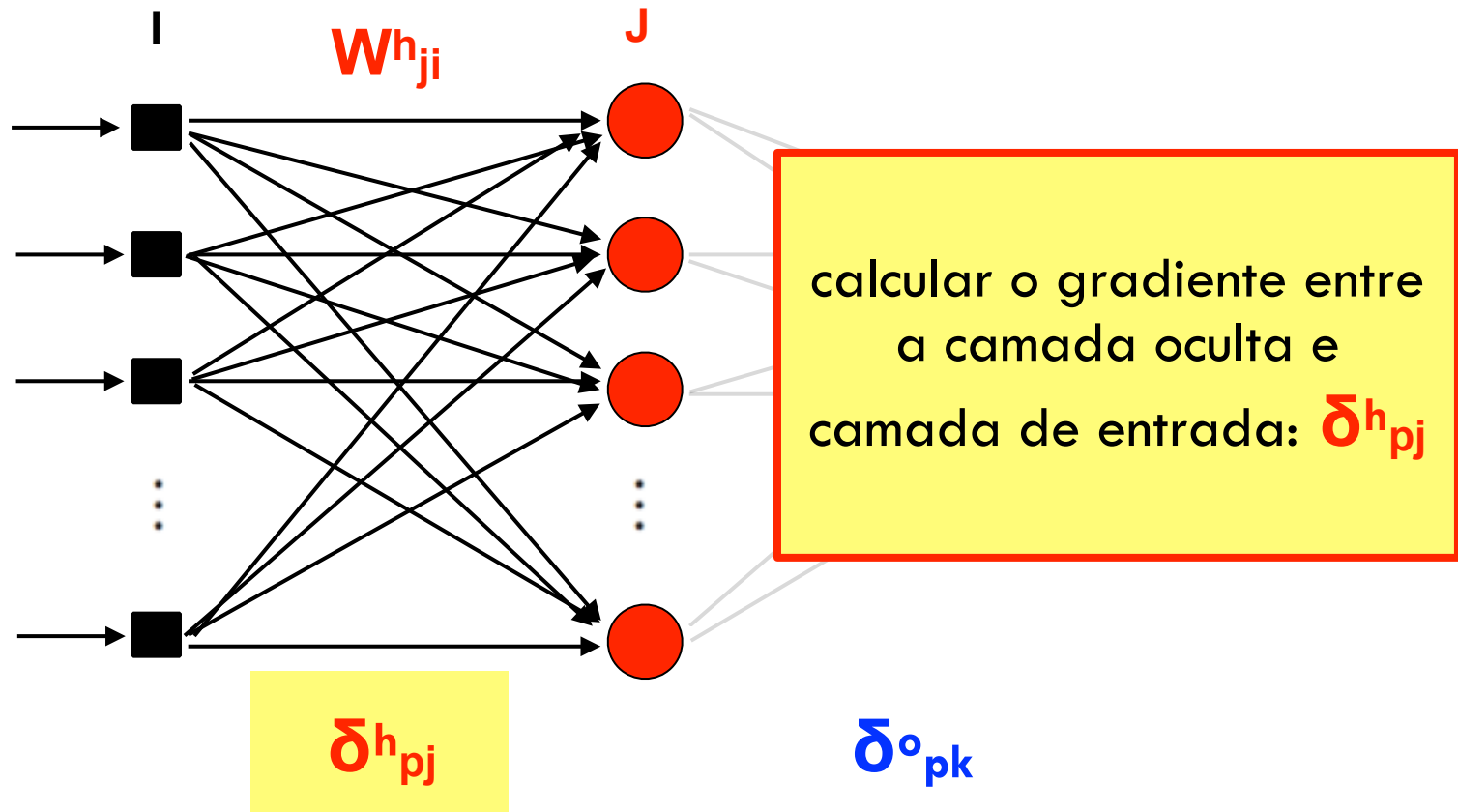
net_{pk}^o : sinal obtido no neurônio k

f_k^o : derivada da função de ativação do neurônio k

Backpropagation



Backpropagation



Backpropagation

- 6 Calcular os termos de erro para as unidades ocultas:

$$\delta_{pj}^h = f_j^h(\text{net}_{pj}^h) \sum_k (\delta_{pk}^o * W_{kj}^o)$$

δ_{pk}^o : termo de erro (gradiente) para o neurônio k da camada de saída

W_{kj}^o : pesos sinápticos que conectam o neurônio j da camada oculta a todos os neurônios k da camada de saída

net_{pj}^h : sinal obtido no neurônio j

f_j^h : derivada da função de ativação do neurônio j

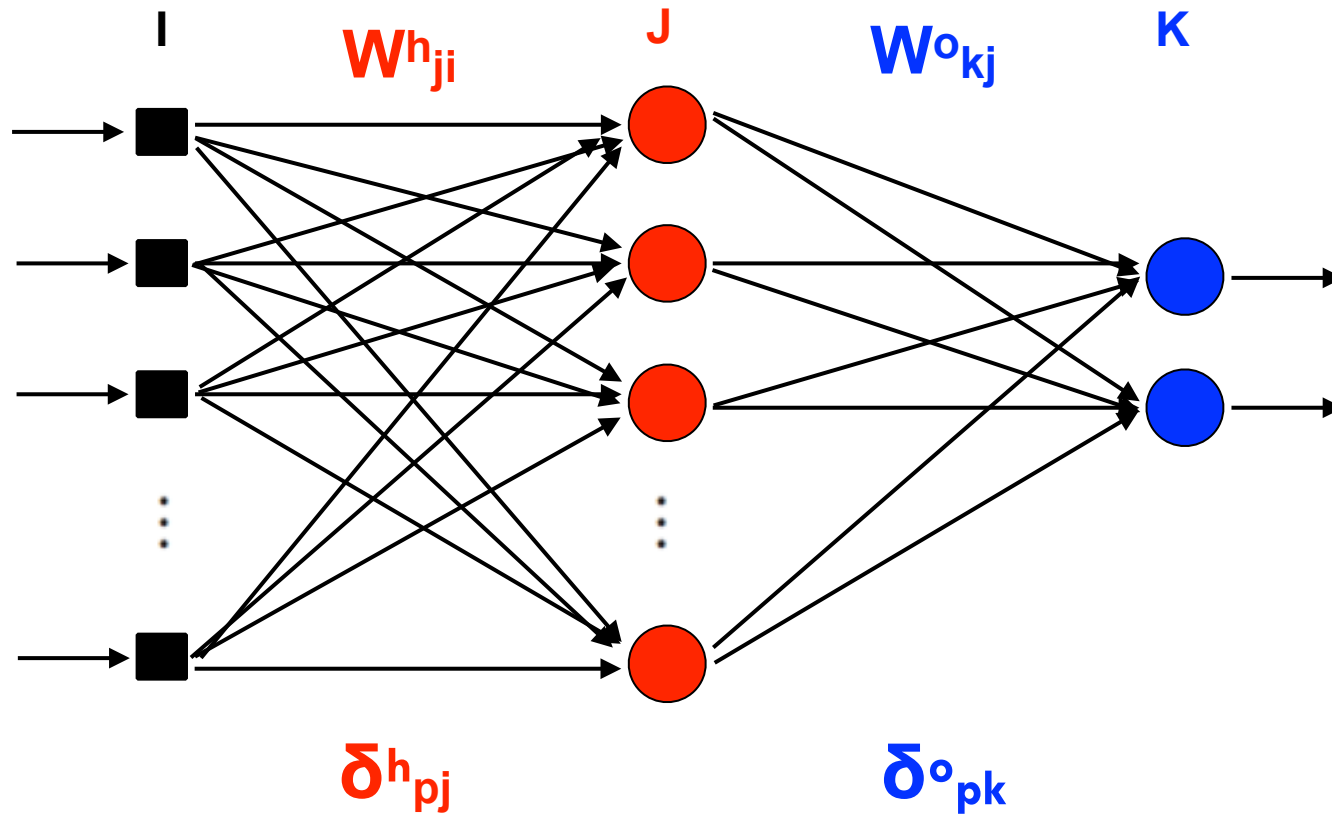
Backpropagation

- 6 Calcular os termos de erro para as unidades ocultas:

$$\delta^h_{pj} = f'^h_j(\text{net}^h_{pj}) \sum_k (\delta^o_{pk} * W^o_{kj})$$

Obs: o erro das unidades ocultas é calculado **ANTES** do ajuste de pesos da camada de saída

Backpropagation



Backpropagation

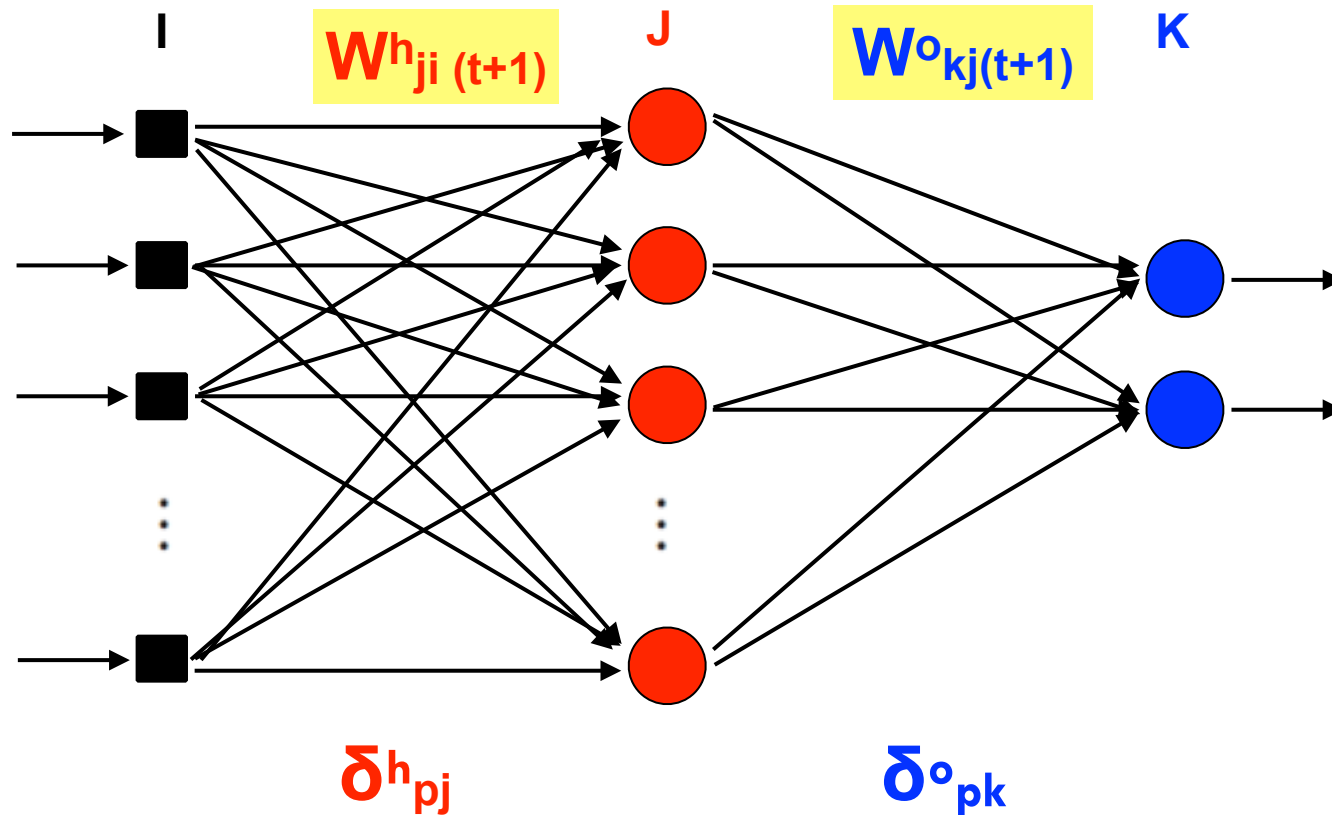
- 7 Atualizar os pesos da camada de saída

$$W^{o}_{kj} (t+1) = W^{o}_{kj} (t) + \eta * \delta^{o}_{pk} * i_{pj}$$

- 8 Atualizar os pesos da camada oculta

$$W^{h}_{ji} (t+1) = W^{h}_{ji} (t) + \eta * \delta^{h}_{pj} * x_{pi}$$

Backpropagation



Backpropagation

- 9 Calcular o erro total da **época**
 - Indica o quão bem a rede está aprendendo.
 - quando for menor que um limiar (***threshold***), parar

Algoritmo resumido

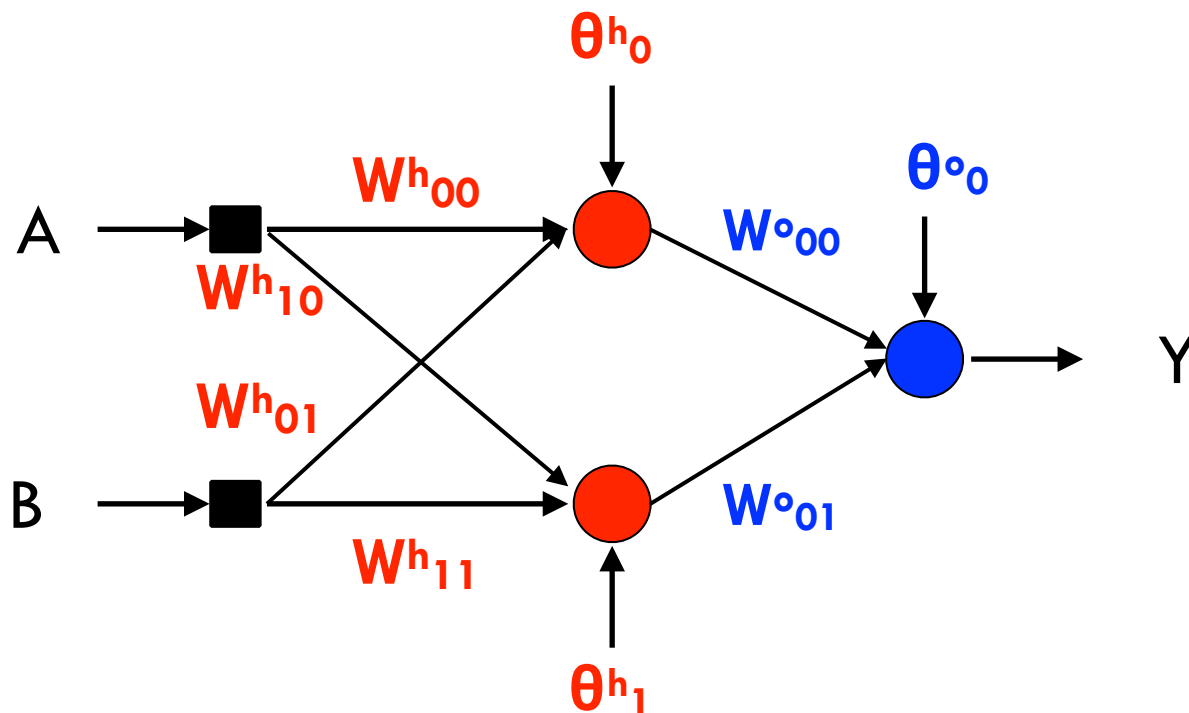
Multilayer Perceptron (MLP):

Início algoritmo

1. Aplicar um **vetor de entrada** para a rede (**X**) e calcular os valores de saída
2. Comparar as **saídas atuais** com as **saídas desejadas** e obter uma medida de **erro**
3. Determinar em qual **direção** (+ ou -) se deve modificar os pesos para minimizar o erro
4. Determinar a **quantidade** para se modificar cada peso
5. Aplicar as **correções aos pesos**
6. Repetir de 1 a 5 com todos os exemplos de treinamento, até que uma margem de erro de treinamento seja atingida

Fim algoritmo

Exemplo



XOR dataset

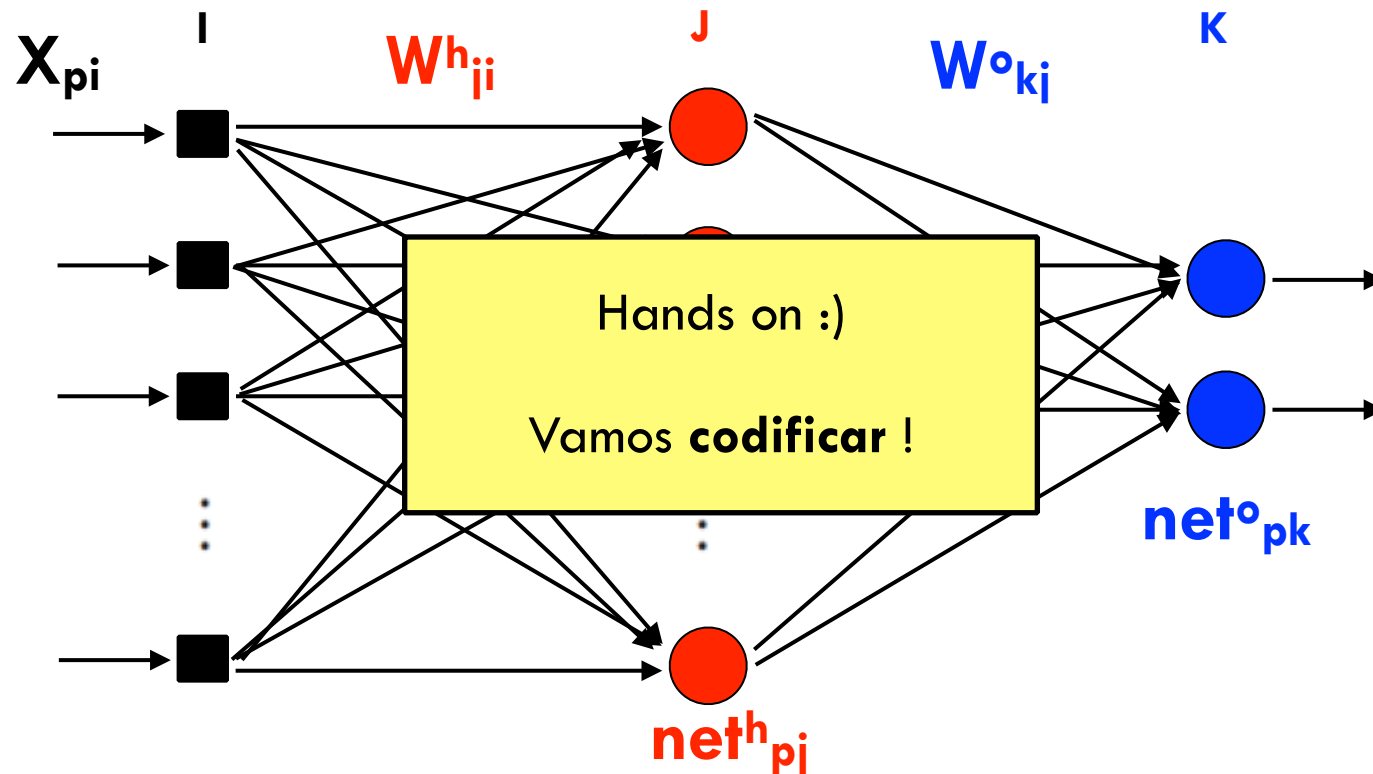
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Exemplo

<i>Iteração</i>	θ^h0	θ^h1	θ^o0	W^h00	W^h10	W^h01	W^h11	W^o00	W^o01
0	0.05	0.06	0.07	0.2	0.15	0.35	0.18	0.10	0.12
1									
2									

- $\eta = 0.2$
- $f(\text{net}) = 1 / (1 + \exp^{-\text{net}})$
- $f'(\text{net}) = \text{net} (1 - \text{net})$

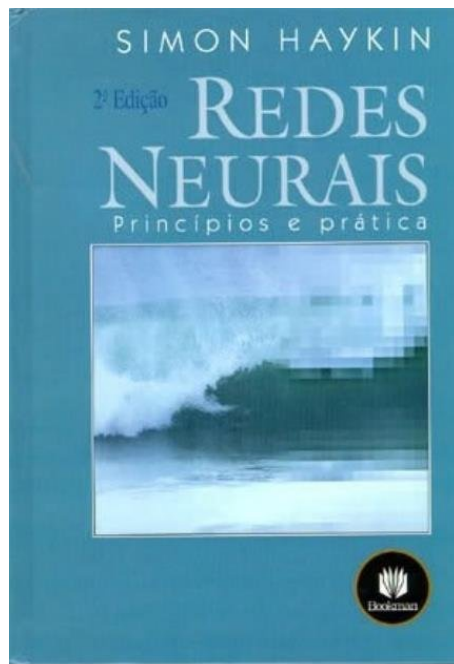
Exercício



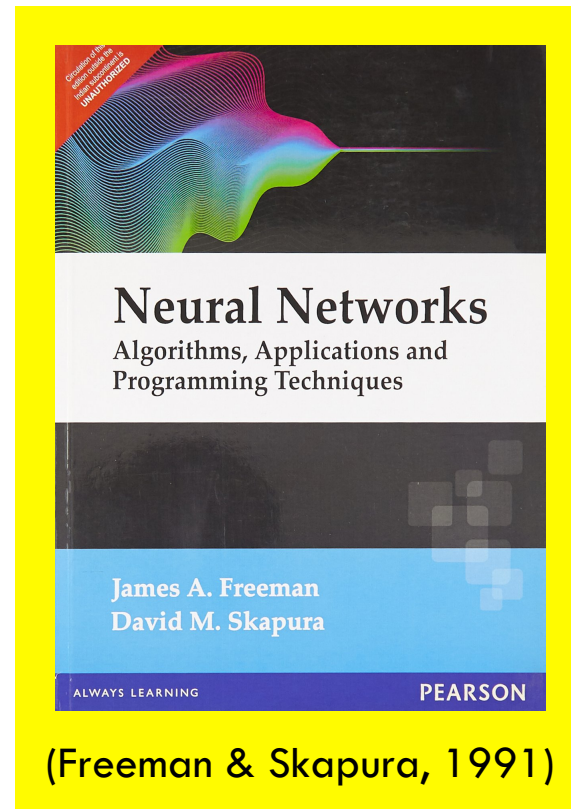
Roteiro

- 1 Introdução
- 2 Multilayer Perceptron
- 3 Formalização
- 4 Treinamento
- 5 Backpropagation
- 6 Referências

Literatura Sugerida

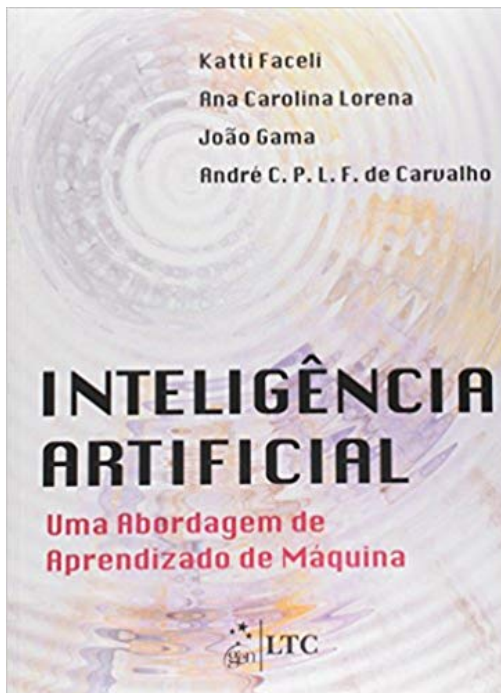


(Haykin, 1999)

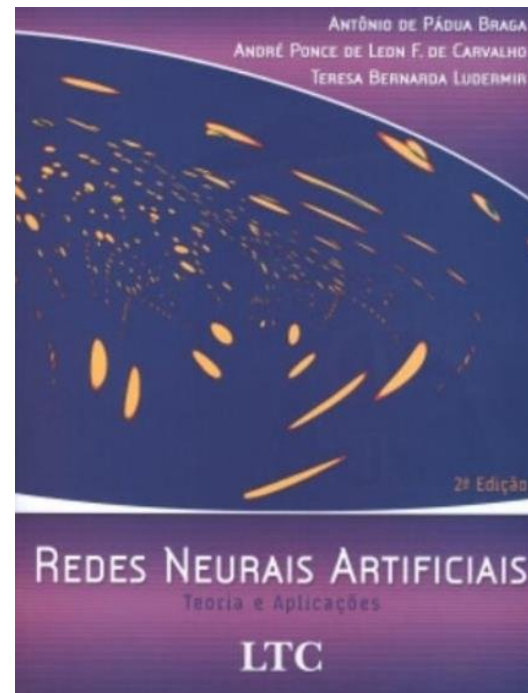


(Freeman & Skapura, 1991)

Literatura Sugerida



[Faceli et al, 2011]



[Braga et al, 2007]



Perguntas?

Prof. Rafael G. Mantovani

rgmantovani@gmail.com